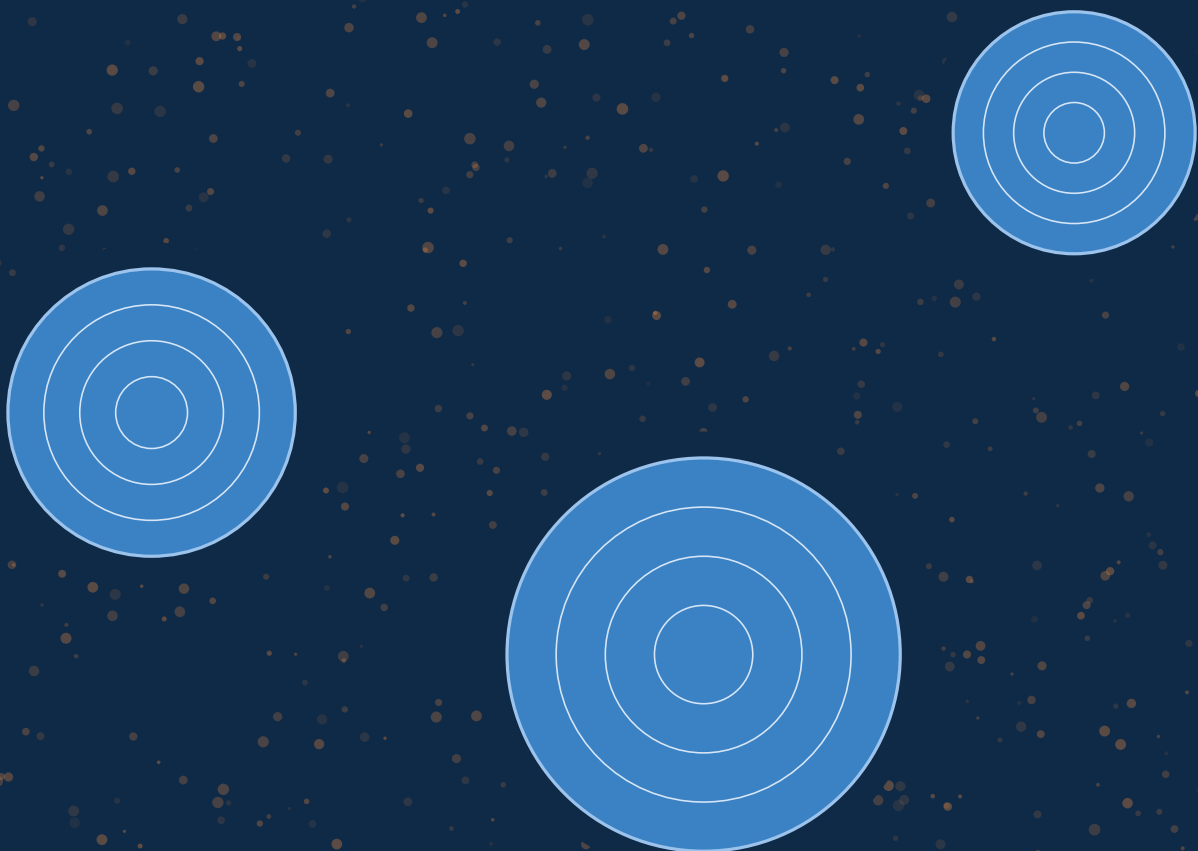


Deterministic Islands in Practice

An Engineer-to-Engineer Guide to the
NEXUS-1 DeterministicIsland Proof of Concept

*Probabilistic core, deterministic shell: how a small C# system keeps a
neural network inside boundaries it cannot learn its way around.*



How to read this guide

This guide is written by an engineer for engineers. It assumes you can read a block diagram, that you have met a probability distribution before, and that you are comfortable with the idea that a program is a model of something real. It does *not* assume that you have trained a neural network, written a hash function or designed a safety system. Everything needed is built up from the ground, one step at a time, using a single small example: the **DeterministicIsland** proof of concept (POC) in this repository.

The POC is a direct companion to *From Stochastic Chaos to Deterministic Certainty* by Grigorios Agathangelidis, Volume III of the NEXUS-1 series. The book develops the theory; the POC is a working, tested, deliberately small machine that puts that theory under load. This guide sits between the two. It explains the POC as **architecture** — what each part is for, what it guarantees, and why it is shaped the way it is — rather than as C# source. Where a precise statement needs code, the guide uses short, language-neutral pseudo-code or the actual data formats written to disk.

Three ways through

- **The quick tour (1 hour).** Read Chapters 1–3, look at Figures 3.1 and 15.1, then read Chapter 25 while running the demo. You will know what every part does and why.
- **The engineer's path (1–2 days).** Read Parts I–IV in order and do the exercises marked *Level 1* and *Level 2* in Chapter 26. This is the intended route for someone starting out in trustworthy AI for critical systems.
- **The reviewer's path.** Start with Part V (what we took from the book, what we changed and why, and the honest boundaries), then the Neural Constitution in Chapter 20, then the test map in Appendix B. That is the evidence a reviewer or regulator would ask for.

Conventions

Marker	Meaning
KEY IDEA	A concept you should be able to explain to a colleague after reading the chapter.
OUR APPROACH	A place where this POC made a decision of its own, beyond or different from the book.
HONEST BOUNDARY	What a mechanism does <i>not</i> guarantee. The book ends every chapter with one; so do we.
TRY IT	A small experiment you can run against the code in a few minutes.
§12.3.1	A section of the book <i>From Stochastic Chaos to Deterministic Certainty</i> .
StaticVault	A name in the code base. You do not need to read the code, but the name tells you where to look.

All numbers in this guide — network outputs, uncertainty bands, hashes, versions — are computed from the same weights and the same rules as the code, by the script that

produced this PDF (docs/source/). If the code changes, re-running the script regenerates the guide with the new numbers.

Contents

How to read this guide	2
Part I — Orientation	6
1 Why this guide, and why this example	7
2 The problem: a correct network that disagrees with itself	8
3 The big picture: probabilistic core, deterministic shell	9
4 Vocabulary	12
Part II — The stochastic core	14
5 A neuron from first principles	15
6 MiniNeuralNetwork, layer by layer	16
7 Dropout: where the randomness comes from	18
8 MC Dropout: turning randomness into a confidence measure	20
9 Frozen Snapshot: the same network, made reproducible	22
Part III — Deterministic Islands	24
10 The Static Vault	25
11 Time: versions, validity and point-in-time answers	27
12 Intra-Vector Routing: facts that point to other facts	28
13 Derived islands: facts computed from facts	29
14 Protection islands and the Shield	31
15 The Arbiter: one control cycle, step by step	32
16 The Causality Lock and Human-in-the-Loop	35
17 Operator override and the limits of human authority	36
Part IV — Trust infrastructure	38
18 Domain events and the audit trail	39
19 Persistence and the hash chain	40
20 The Neural Constitution	42
21 The Web API	43
Part V — Our approach	44
22 From book to code: a traceability map	45
23 Design decisions and their rationale	46
24 Honest boundaries	47

Part VI — Hands-on	48
25 The eleven scenarios, explained	49
26 Exercises: a graded path into the field	51
27 Where to go next	52
Appendices	55
A Code map	56
B Test map	57
C Rebuilding this guide	57

PART I

Orientation

Why a neural network cannot be trusted on its own in a control loop, what the book proposes instead, and the whole POC on one page.

1 Why this guide, and why this example

A valve, two sensors, one small neural network and a set of rules it is not allowed to break. That is enough to learn the core ideas of trustworthy AI for critical infrastructure.

Every plant engineer knows the difference between a *measurement* and a *guess*. A measurement is reproducible: same conditions, same instrument, same number. A guess may be excellent on average and still wrong on the one occasion that matters. Modern AI is, at its core, an extremely good guesser. It is trained on data, it generalises, and it is flexible in ways no hand-written rule ever will be. It is also — by construction — stochastic: random weight initialisation, dropout, and sampling at inference time are not bugs but features that make the model learn and generalise (book, §12.2.1).

The book's central claim is that in critical systems this flexibility is only acceptable inside boundaries that are **not** learned: reproducible, auditable, certifiable reference points it calls **Deterministic Islands**. The AI may reason freely in the open sea; the hard facts it must respect — a relief setpoint, a maximum valve opening, a SCRAM condition — live on islands where identical questions always receive identical answers.

The POC turns that claim into a small control system. A **stochastic core** (a tiny neural network) proposes how far to open a valve based on temperature and pressure. A **deterministic shell** decides whether that proposal may reach the actuator, replaces it when a safety rule applies, and hands the decision to a human when it cannot be sure. Everything it does is recorded so that, afterwards, anyone can answer the question *which rule, which version, approved by whom, produced this command?*

Why such a small example?

Because small systems can be understood completely. The network has two inputs, four hidden neurons and one output: 17 numbers in total. You can compute its output by hand (we do so in Chapter 6). Its randomness comes from exactly one place — dropout on four neurons — so there are only 16 possible internal configurations, and we can enumerate all of them (Chapter 7). The demo vault holds eleven facts. The arbiter has six possible decisions. There are eleven scenarios and one hundred automated tests.

Nothing in this example is realistic in scale, and that is the point. Every mechanism in it scales to a real plant or a real language model — hashing, versioning, pointer resolution, precedence, uncertainty estimation, escalation, audit — but here each one fits on a page. Once you understand why each is there, adding scale is an engineering exercise, not a conceptual leap.

KEY IDEA · What you should be able to do after this guide

Explain why a correctly working neural network can give two different answers to the same question, and why that matters in a control loop.

Design a deterministic boundary around a stochastic component: which facts it holds, how they are versioned, how the system decides who is in charge, and when it must stop and ask a human.

Read the POC's audit trail and reconstruct exactly why each command was issued.

What the POC is not

It is not a reactor controller, a certified safety system, or a model of any real plant. The numbers (8 bar, 90 °C, 75%) are illustrative. The network's weights were chosen by hand to produce a plausible, smooth response surface, not trained on data. The disclaimer at the end of the README applies throughout: this code must not be used to operate or make safety decisions in a real facility. What it offers instead is a faithful, testable architecture that you can study, break and extend.

2 The problem: a correct network that disagrees with itself

The book's §12.2 in one sentence: nothing has malfunctioned, and yet the same input produces different outputs.

Run the POC's network three times on the same reading — 60 °C and 5 bar — and you may get 37.2%, 29.7% and 50.0% valve opening. Nothing is broken. The network uses **dropout**: on each call, each of its four hidden neurons is switched off with probability 0.2. Which neurons survive is a random draw, and each draw is a slightly different network. The book traces three separate sources of this behaviour in real systems — random initialisation, dropout, and temperature sampling in language models — and makes the same observation about all of them: the randomness is deliberate and useful everywhere except for a specific class of questions (§12.2.1).

Why a plant cannot live with this

Consider two operators on two shifts who both ask the system for the maximum permitted discharge pressure of a pump and receive two different numbers. The book is blunt about it: they have not encountered a minor inconsistency; they have encountered a system that cannot be trusted to state a fact (§12.2.3). In a control loop the problem is sharper still. If the command sent to a valve depends on a coin flip, then the plant's behaviour depends on a coin flip, and after an incident no one can reproduce what happened.

There are really three distinct requirements hiding here, and the POC addresses each with a different mechanism:

Requirement	Question it answers	Mechanism in the POC
Reproducibility	Will the same question get the same answer tomorrow, on another server?	Static Vault (hash-addressed facts); Frozen Snapshot (locked inference)
Boundedness	Can the AI ever command something outside the certified range?	Shield; protection islands; priority rings; SCRAM
Accountability	Afterwards, can we say exactly which rule and which approval produced a command?	Versioned facts with approvers; domain events; audit trail; hash-chained storage

Two kinds of question

The book's worked example in §12.5 separates two questions an operator might ask. “*What is the maximum allowed pressure?*” has one correct answer that a human engineer has verified; it should never be generated. “*What do you recommend for the current situation?*” has no fixed answer and genuinely needs the model's flexible reasoning — but if that recommendation mentions a limit, the limit itself must come from the verified source, not from the model.

The POC's control loop has exactly the same structure. *How far should the valve open right now?* is a recommendation, and the network is allowed to make it. *What is the relief setpoint, the maximum valve opening, the containment position?* are facts, and the network is never asked for them. The whole architecture is the machinery that keeps these two kinds of answer apart and decides, cycle by cycle, which one governs the actuator.

KEY IDEA · Randomness is not the enemy

The goal is not to remove stochasticity from the system; that would throw away the reason for using AI at all. The goal is to put it in its place: free where flexibility helps, excluded where a guarantee is required, and *measured* everywhere in between. The POC even uses the network's randomness as a signal: the spread of its answers tells us how much to trust it (Chapter 8).

3 The big picture: probabilistic core, deterministic shell

One figure, one control cycle, one query path. Everything later in the guide is a zoom into a box on this page.

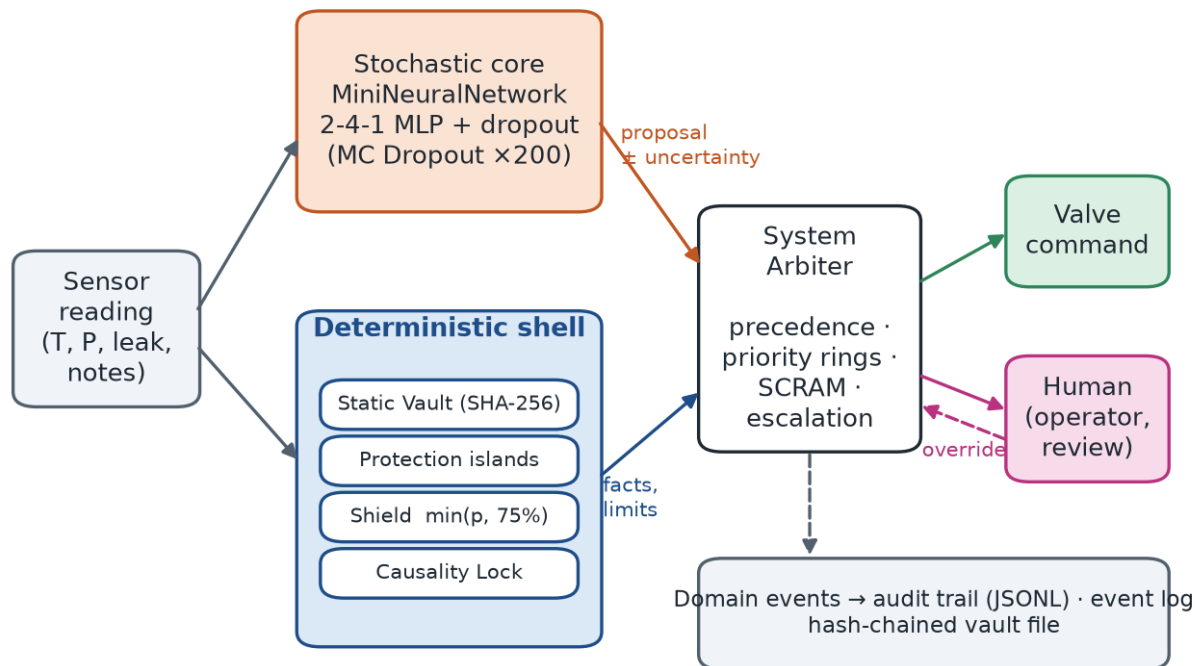


Figure 3.1 — The POC's architecture. The stochastic core proposes; the deterministic shell holds the facts and the rules; the Arbiter decides; humans can intervene; everything is recorded.

A **sensor reading** enters the system: temperature, pressure, a radiation-leak flag and free-text operator notes. Two things happen in parallel. The **stochastic core** computes a proposed valve opening together with an estimate of how uncertain that proposal is. The **deterministic shell** looks up, in the Static Vault, every safety fact valid at this moment and builds the protection islands from them.

The **System Arbiter** then decides. If a safety island is triggered, the island's command wins, and if two equally important islands disagree, the system fails safe with a SCRAM. If no island is triggered, the AI's proposal may be used — but only if the operator has not asked for a deterministic answer, only if the AI is confident enough, and only after the Shield caps it to the certified maximum. When none of these conditions allows a safe automatic answer, the Arbiter does not guess: it escalates to a human.

Every decision publishes a **domain event**. Listeners write an audit trail and an event log; every new version of a safety fact is appended to a hash-chained file. Nothing that influenced a command is lost.

Two paths through the shell

	Control path — Execute(reading)	Query path — Answer(query)
Input	Sensor reading, optional operator override	A question in words, optional candidate answer
Output	A valve command, a SCRAM, or an escalation	A deterministic fact, a fall-through, or an escalation

	Control path — Execute(reading)	Query path — Answer(query)
Book section	§17.4 Shielding, §12.4.3 precedence	§12.4.2 the Arbiter, §12.5 worked example
Example	95 °C, 4 bar → Structural island → 60%	“high pressure relief setpoint in bar” → 8.0 (v1)

Most of this guide follows the control path, because that is where the interesting interactions are. The query path reuses the same vault and resolver and is covered in Chapters 10–13 and 21.

The layers

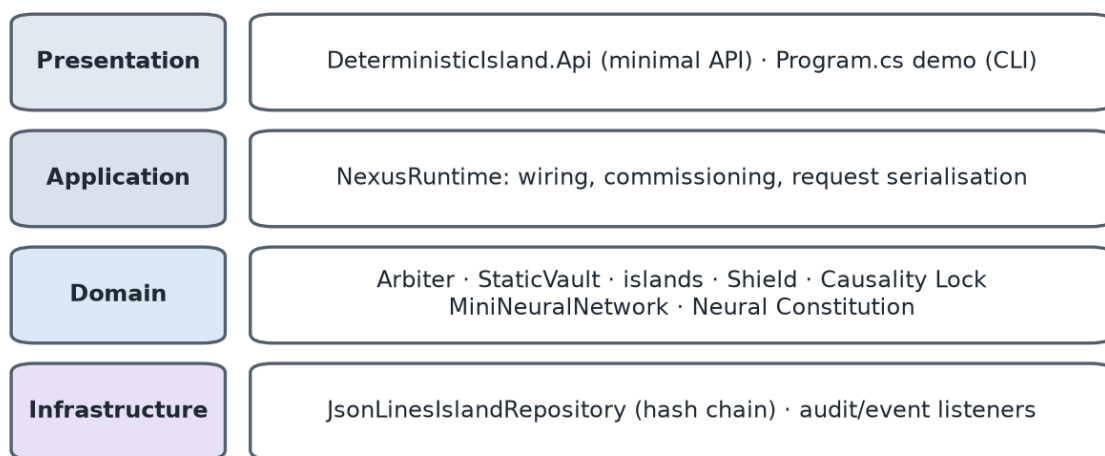


Figure 3.2 — The same system seen as layers (book, Chapters 20–23). The domain layer holds all the rules; the outer layers only wire, store and expose them.

The layering follows the book's Domain-Driven Design chapters. The **domain** layer contains every rule that matters for safety and has no knowledge of files, HTTP or consoles. The **infrastructure** layer stores and logs. The **application** layer wires the pieces together once. The **presentation** layer — a console demo and a small Web API — is thin by design: as the book puts it, the endpoint has no way to bypass the Arbiter, because the Arbiter is the only path to an answer (§23.5).

4 Vocabulary

The book calls this the Ubiquitous Language (§20.4.1): words that mean the same thing to the plant engineer, the software engineer and the reviewer.

Term	Meaning in this POC
Stochastic core	The neural network that proposes valve openings. Allowed to be flexible; never trusted with facts.
Deterministic Island	A safety fact (or rule built from facts) whose answer is identical for identical questions, pre-verified and approved by a human.
Static Vault	The store of deterministic facts, addressed by SHA-256 of the normalised question; append-only and versioned.
ILMVector	The book's unit of knowledge: an id, an embedding (here: one number) and an optional Determinism block.
Determinism block	Metadata that makes a vector part of an island: IsDeterministic, Version, ValidFrom/ValidTo, ApprovedBy, PointerTo, DerivedFrom.
Version	One entry in the append-only history of a fact (v1, v2, ...). Old versions are closed, never edited.
Pointer (Intra-Vector Routing)	A vault entry that defers to another vector instead of holding a value.
Derived island	A fact computed as a pure function (e.g. minimum) of other facts, recomputed when they change.
Protection island	A sensor-triggered interlock (pressure relief, radiation containment, thermal protection) built from vault facts.
Priority ring	The rank of a protection island: CriticalSafety > Structural > Operational.
Shield	The bound applied to every AI proposal: min(proposal, maximum safe opening).
Frozen Snapshot	A configuration that makes the network bitwise reproducible: locked weights, runtime, seed, CPU-only.
MC Dropout	Running the network many times with dropout on and using the spread as an uncertainty estimate.
Causality Lock	Engaged when the operator explicitly requests a deterministic answer; forbids a stochastic one.
Human-in-the-Loop	Escalation: the system refuses to act on its own answer and hands the decision to a person.
Operator override	A command from a named operator with a reason. Beats the AI, never a safety island.
SCRAM	Fail-safe shutdown (valve to 0%) when equally ranked safety islands disagree.
System Arbiter	The component that combines all of the above into one decision per cycle.
Domain event	A record of something that happened (IslandAdded, IslandTriggered, ControlDecisionMade).

Term	Meaning in this POC
Audit record	One line per control cycle describing every input to the decision and the decision itself.
Neural Constitution	The list of rules the system never violates, each linked to its mechanism and its tests.

PART II

The stochastic core

A neural network small enough to compute by hand: where its answers come from, where its randomness comes from, how to measure its confidence, and how to freeze it.

5 A neuron from first principles

Before the network, one neuron. If you understand a weighted sum and a squashing function, you understand everything the stochastic core does.

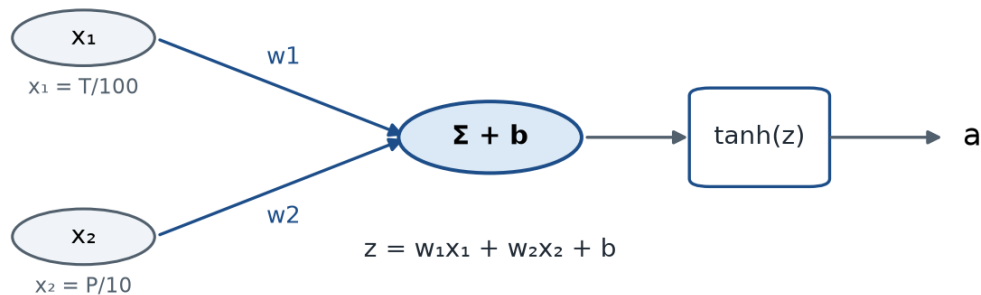


Figure 5.1 — One hidden neuron of the POC: a weighted sum of the two scaled inputs plus a bias, passed through $\tanh$.

A neuron does two things. First it computes a **weighted sum** of its inputs plus a constant called the **bias**: $z = w_1x_1 + w_2x_2 + b$. The weights say how much each input matters and in which direction; the bias shifts the point at which the neuron starts to respond. This part is ordinary linear algebra — the same operation as a weighted average or a linear regression (book, Chapter 4).

Second, it passes z through an **activation function**. Without this step a network of any depth would collapse into one big linear function and could never represent a curved response (the XOR problem of the book's Chapter 5 is the classic proof). The POC uses two activations:

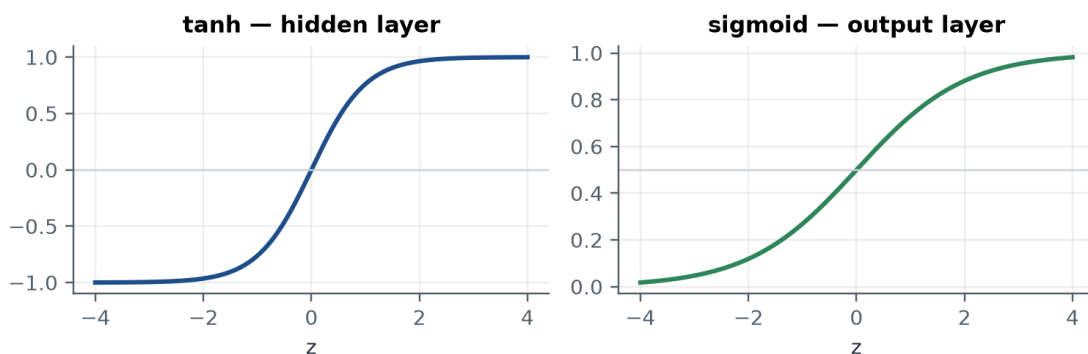


Figure 5.2 — The two activation functions used in the POC.

- **tanh** in the hidden layer. It maps any real number to $(-1, 1)$, is roughly linear near zero and saturates for large $|z|$. Positive and negative outputs let hidden neurons push the final answer either way.
- **sigmoid** $\sigma(z) = 1 / (1 + e^{-z})$ at the output. It maps any real number to $(0, 1)$, which is exactly the range of a valve opening from fully closed to fully open. The value before the sigmoid is called the **logit**.

KEY IDEA · Scaling the inputs

The network never sees raw engineering units. Temperature is divided by 100 and pressure by 10, so a typical reading lands in the range 0 to 1. Without this, a pressure of 8 and a temperature of 80 would enter the weighted sum with wildly different magnitudes and the neuron would be dominated by whichever unit happens to produce bigger numbers. Input scaling is the first thing to check when a network behaves strangely.

6 MiniNeuralNetwork, layer by layer

Two inputs, four hidden neurons, one output, seventeen numbers. We compute an answer by hand.

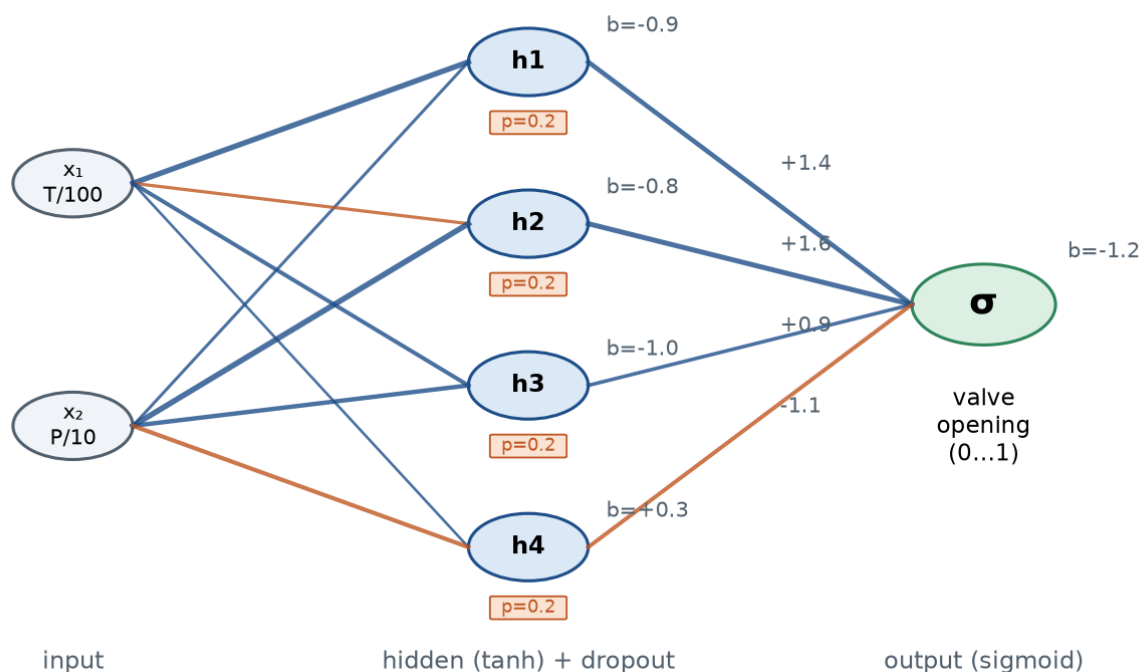


Figure 6.1 — The 2-4-1 network. Blue connections have positive weights, orange negative; thickness is proportional to $|w|$. Each hidden neuron can be dropped with $p = 0.2$.

The MiniNeuralNetwork is a **multilayer perceptron** (MLP) with the shape 2-4-1. Its entire knowledge is the seventeen numbers in the table below: eight hidden weights, four hidden biases, four output weights and one output bias. They were set by hand to give a smooth, plausible response: the valve opens more as pressure rises and, more gently, as temperature rises. In a real system these numbers would come from training (book, Chapter 7). Nothing in the architecture around the network depends on how they were obtained.

Neuron	w (temperature)	w (pressure)	bias	w to output
h1	+1.8	+0.6	-0.9	+1.4
h2	-0.7	+1.9	-0.8	+1.6
h3	+1.1	+1.3	-1.0	+0.9

Neuron	w (temperature)	w (pressure)	bias	w to output
h4	+0.4	-1.2	+0.3	-1.1
output			-1.2	

A worked forward pass: 60 °C, 5 bar

Step 1 — scale. $x_1 = 60/100 = 0.6$, $x_2 = 5/10 = 0.5$.

Step 2 — hidden layer. For each neuron compute $z = w_1x_1 + w_2x_2 + b$ and $a = \tanh(z)$:

Neuron	z	a = tanh(z)	contribution if kept: w·a / 0.8
h1	$-0.9 + 1.8 \times 0.6 + 0.6 \times 0.5 = +0.4800$	+0.4462	+0.7809
h2	$-0.8 - 0.7 \times 0.6 + 1.9 \times 0.5 = -0.2700$	-0.2636	-0.5272
h3	$-1.0 + 1.1 \times 0.6 + 1.3 \times 0.5 = +0.3100$	+0.3004	+0.3380
h4	$+0.3 + 0.4 \times 0.6 - 1.2 \times 0.5 = -0.0600$	-0.0599	+0.0824

Step 3 — output. With all four neurons kept, the logit is the output bias plus the four contributions:

$$\text{logit} = -1.2 + 0.7809 - 0.5272 + 0.3380 + 0.0824 = -0.5259$$

$$\sigma(-0.5259) = 1 / (1 + e^{0.5259}) = 0.3715$$

The network proposes a valve opening of **37.1%**.

Why the division by 0.8? That is **inverted dropout**, explained in the next chapter. It keeps the *expected* contribution of each neuron the same whether or not dropout is active.

TRY IT · Compute a second reading yourself

Repeat the three steps for 85 °C and 7 bar. You should find hidden activations of about 0.782, -0.065, 0.688 and -0.197, a logit of 1.084 with all neurons kept, and an opening of 74.7%. Chapter 8 explains why this reading nevertheless ends up escalated to a human.

What the network has learned: the policy surface

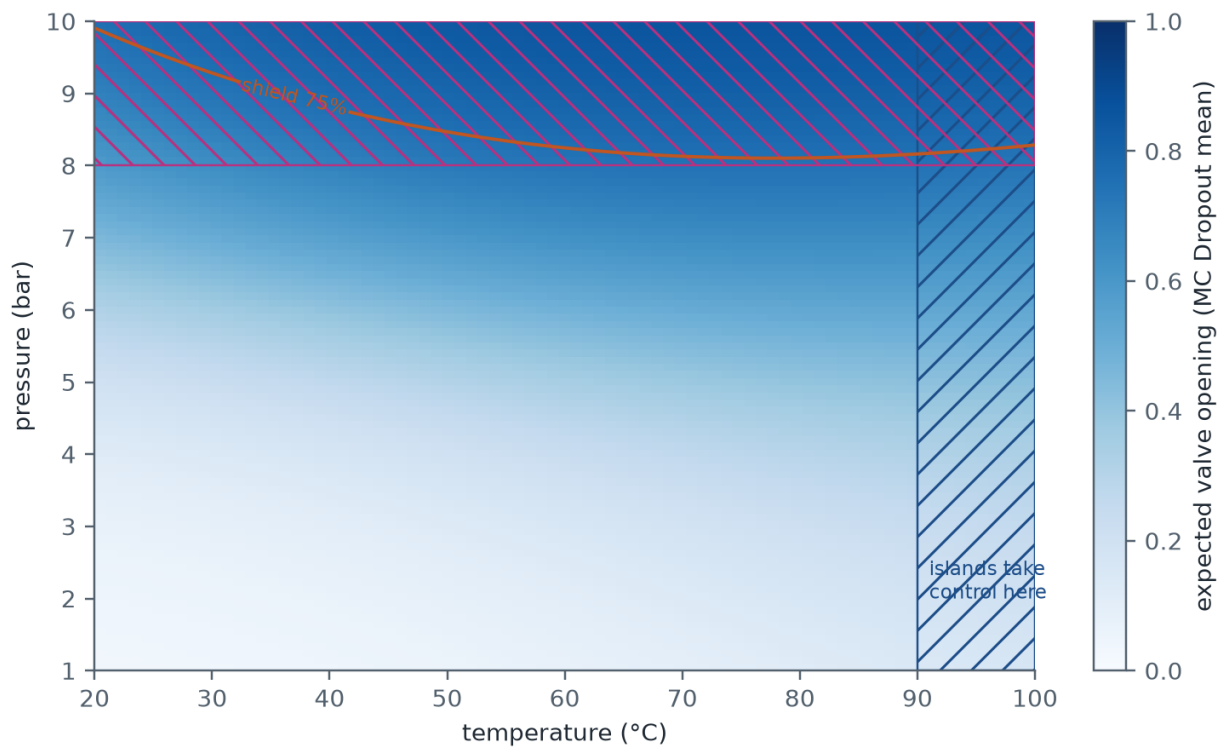


Figure 6.2 — Expected valve opening over the whole operating envelope. Hatched regions are where protection islands take control and the network's answer is not used at all. The orange contour marks the Shield's 75% cap.

Plotting the network's expected output over every temperature and pressure gives its **policy surface**. Two things are worth noticing. First, it is smooth: nearby readings produce nearby answers, which is what we want from a controller. Second, in the upper band — high pressure — the network's own answer climbs above 75%. This is precisely where the architecture stops trusting it: above 8 bar the pressure-relief island takes over, and anywhere else the Shield caps the command. The network is free to be wrong in those regions, because its answer is never applied there.

7 Dropout: where the randomness comes from

Four coin flips per call. Sixteen possible networks. One distribution of answers.

Dropout was invented as a training trick (book, §7.6.4): randomly switching off neurons during training prevents the network from relying too heavily on any single one, which improves generalisation. Normally it is turned off at inference. The POC deliberately leaves it **on**, for two reasons. It reproduces, in the smallest possible system, the exact problem of §12.2 — a correct model that answers the same question differently — and it gives us, for free, a way of measuring the model's uncertainty (Chapter 8).

On every call, each hidden neuron is independently dropped with probability $p = 0.2$. A dropped neuron contributes nothing; a kept one contributes $w \cdot a / (1 - p)$. The division by

$(1 - p) = 0.8$ is **inverted dropout**: because a neuron is present only 80% of the time, scaling its contribution up by $1/0.8$ keeps its *average* contribution equal to $w \cdot a$.

Enumerating every possible network

Four neurons, each kept or dropped, give $2^4 = 16$ possible masks. The probability of a mask with k kept neurons is $0.8^k \cdot 0.2^{4-k}$. For the reading 60 °C, 5 bar we can therefore compute the *exact* distribution of the network's answer:

Mask (h1 h2 h3 h4)	Probability	Output	Mask	Probability	Output
1111	0.4096	0.3715	0110	0.0256	0.1995
1110	0.1024	0.3524	0101	0.0256	0.1618
1101	0.1024	0.2965	0011	0.0256	0.3144
1011	0.1024	0.5003	1000	0.0064	0.3967
0111	0.1024	0.2130	0100	0.0064	0.1509
1100	0.0256	0.2796	0010	0.0064	0.2969
1010	0.0256	0.4797	0001	0.0064	0.2465
1001	0.0256	0.4166	0000	0.0016	0.2315

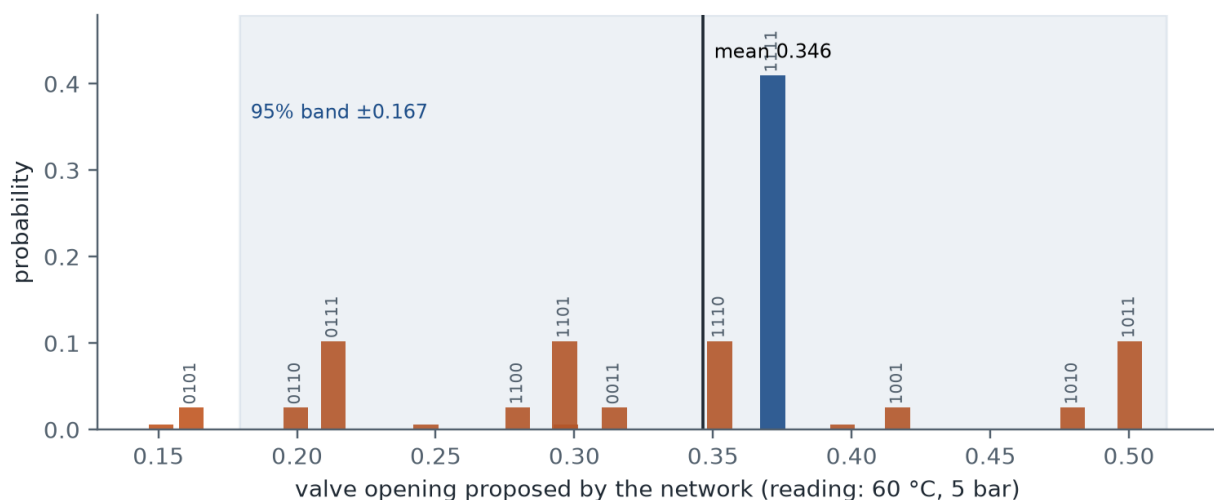


Figure 7.1 — The exact distribution of the network's answer at 60 °C, 5 bar. The blue bar is the 'all neurons kept' network (41% of calls); every other bar is a network with one or more neurons dropped.

Read the figure as an engineer would read a histogram of repeated measurements. The answers range from 15.1% to 50.0%. The most likely single answer (all neurons kept) is 37.1%, but it occurs only 41% of the time. The expected value is 34.6% and the standard deviation is 8.5 percentage points. If this network drove the valve directly, the valve could swing between 15% and 50% from one cycle to the next with the plant in a perfectly steady state.

KEY IDEA · Stochastic ≠ broken

Every one of these sixteen networks is a legitimate instance of the same model. The spread is not noise added from outside; it is the model's own internal disagreement. That is exactly why it can be used as a measure of confidence in the next chapter.

8 MC Dropout: turning randomness into a confidence measure

Ask the same question 200 times. The average is the answer; the spread is how much to trust it. (Book, §13.6.)

Monte Carlo (MC) Dropout is the book's practical alternative to full Bayesian neural networks (§13.6). Instead of one forward pass, run many with dropout left on, and treat the resulting sample as an empirical distribution. The POC runs **200 passes** per control cycle and computes:

```
mean      = average of the 200 outputs          -> the AI's proposal
s         = sample standard deviation (divide by n - 1)
halfWidth = 1.96 * s                            -> the 95% band is mean ± hal
          fWidth
interval  = [max(0, mean - halfWidth), min(1, mean + halfWidth)]
```

The 1.96 comes from the normal distribution: about 95% of a normal population lies within 1.96 standard deviations of the mean. The book's worked example uses exactly this recipe on a remaining-useful-life prediction (44.2 ± 4.2 days); the POC applies it to a valve opening.

Where the network is sure, and where it is not

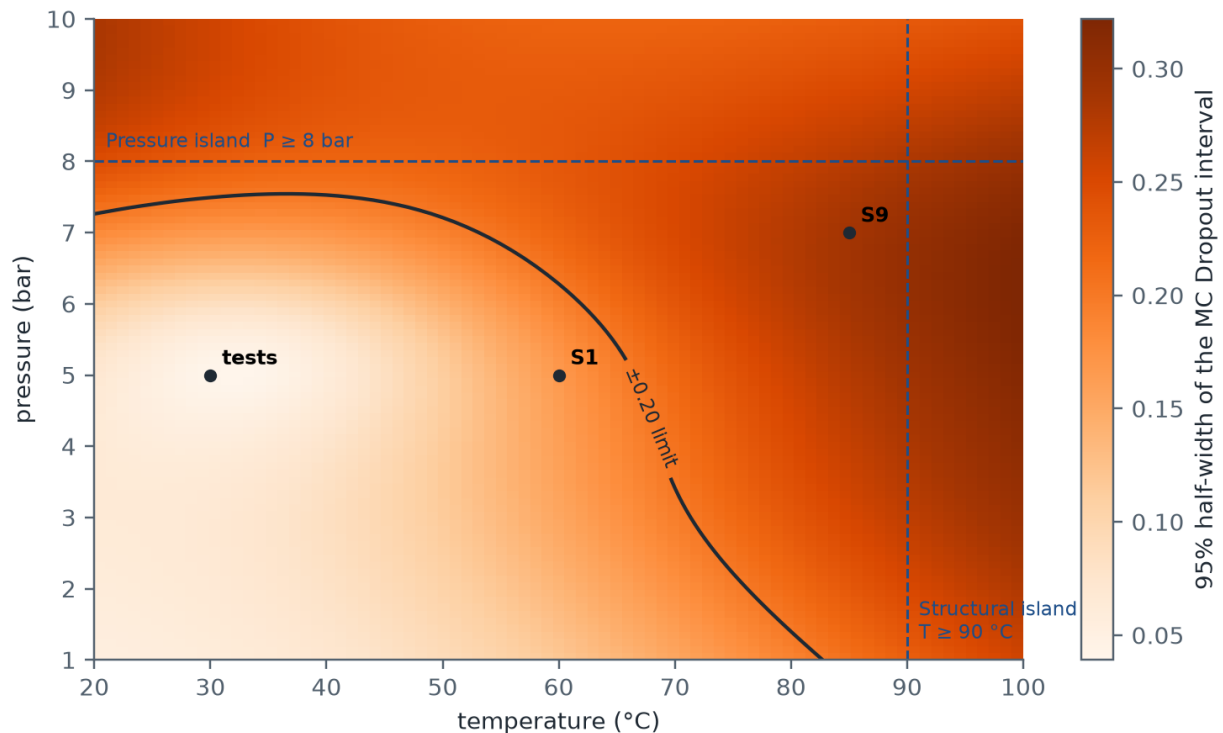


Figure 8.1 — 95% half-width of the MC Dropout interval over the operating envelope. Inside the black contour the network is within the vault's ± 0.20 limit; outside it the command is escalated to a human. S1 and S9 are demo scenarios 1 and 9.

Reading	Mean	Std. dev. s	95% half-width	Limit	Outcome
30 °C, 5 bar (tests)	0.232	0.020	± 0.040	± 0.20	Approved
60 °C, 5 bar (scenario 1)	0.346	0.085	± 0.167	± 0.20	Approved
85 °C, 7 bar (scenario 9)	0.643	0.147	± 0.288	± 0.20	Escalated

These values are exact (computed over the sixteen masks). With 200 random passes the POC's estimate of the half-width wobbles by about ± 0.01 – 0.02 , which is why the tests use readings well inside or well outside the limit. The map shows a pattern that is common in real models: the network is confident where all its neurons agree — low temperature and moderate pressure — and uncertain where neurons with opposite signs are both strongly active, which is the hot, high-pressure corner.

The threshold is a fact, not a model setting

The permitted half-width, ± 0.20 , is not a constant in the code. It is a Static Vault fact — “maximum ai prediction uncertainty as 95% half-width” — with a version and an approver. This follows the book's advice in §16.9 about anomaly detection: the *score* is legitimately stochastic and model-dependent, but the *threshold* at which a human must look is an engineering decision that no amount of retraining should move quietly. Changing it requires a new version in the vault, exactly like changing a relief setpoint.

OUR APPROACH · Uncertainty escalation is our addition

The book introduces MC Dropout as an evaluation technique (§13.6) and argues for resolving alarm thresholds through Deterministic Islands (§16.9). Using the interval *inside the control loop* — refusing to apply a command whose 95% band is wider than a vault limit and escalating instead — is this POC's own step. It became Rule 6 of our Neural Constitution (Chapter 20).

HONEST BOUNDARY · What MC Dropout does not tell you

It measures the model's disagreement with itself, not its distance from the truth. A network that is confidently wrong — for example on data unlike anything it was trained on — can show a narrow band. That is why uncertainty is one guard among several, not a replacement for islands and the Shield.

9 Frozen Snapshot: the same network, made reproducible

Sometimes you need the model's reasoning and a guarantee that it will give the same answer tomorrow. The book's §12.3.2 lists exactly what must be locked.

A Static Vault answers questions whose answers can be listed in advance. Some behaviour, however, must come from the model itself — and still be reproducible bit for bit. The **Frozen Snapshot** achieves this by removing every remaining degree of freedom from inference. The book names four, and the POC's `FrozenSnapshotConfig` has exactly those four fields:

Locked item	Why it matters	How the POC enforces it
Model weights	Different weights, different answers.	SHA-256 of all 17 weights; the snapshot is rejected if it does not match.
Runtime version	Math libraries can round differently between releases.	The .NET runtime version is recorded; a mismatch rejects the snapshot.
Random seed	Dropout masks must be the same every time.	A new random generator with the fixed seed is created for <i>every</i> call.
CPU-only inference	Parallel sums finish in varying order (see below).	Single-threaded, fixed summation order; a non-CPU config is rejected.

Why the order of a sum matters

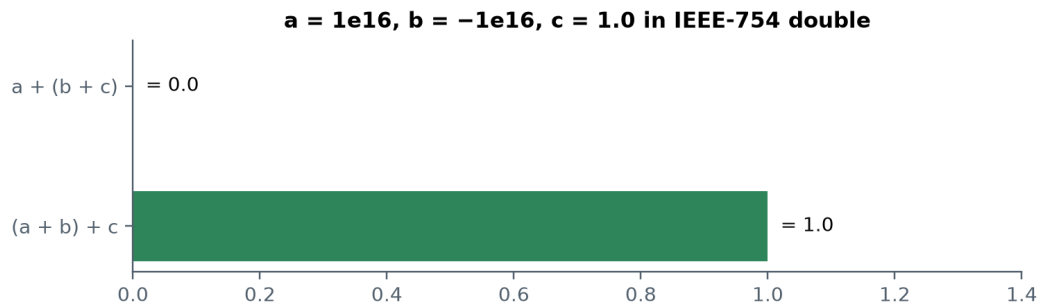


Figure 9.1 — The book's §24.2.2 example: in IEEE-754 double precision, $(a + b) + c$ and $a + (b + c)$ give different answers.

Floating-point addition is not associative. With $a = 10^{16}$, $b = -10^{16}$ and $c = 1$, computing $(a + b) + c$ gives 1, but $a + (b + c)$ gives 0, because 1 is below the precision available at 10^{16} and disappears in $b + c$. A GPU sums the many products of a neural network in parallel, in an order that depends on which thread finishes first; the result can therefore differ in the last bits from run to run. A single-threaded CPU loop fixes one order. In the POC this is visible in the code as a plain loop over the hidden neurons with a comment pointing to §24.2.2.

When the snapshot is used

The Arbiter uses the frozen core only when the operator has engaged the **Causality Lock** (Chapter 16) and no island decides the cycle. In that case the ordinary stochastic answer is forbidden, but a frozen, reproducible answer is acceptable: it sits third in the book's precedence order, after the Static Vault and a routed vector (§12.4.3). Scenario 6 shows it: the same reading twice, 41.7% both times, identical to the last bit.

HONEST BOUNDARY · Reproducible is not the same as correct

A Frozen Snapshot guarantees that the model will repeat itself, not that it is right. The book's Honest Boundary in §24.3 adds a second cost: single-threaded CPU inference can be one to two orders of magnitude slower for a large model. It is a tool for the narrow class of queries that genuinely need reproducibility, not a general setting.

PART III

Deterministic Islands

The facts the AI is never asked for, how they are stored and versioned, how they are combined, and how one component — the Arbiter — decides who is in charge on every cycle.

10 The Static Vault

The strongest guarantee is the simplest mechanism: answer = Vault[SHA-256(normalised query)]. (Book, §12.3.1.)

For a known, finite set of safety-critical facts the book's advice is to bypass the neural network entirely. The **Static Vault** maps a question directly to a stored, pre-verified answer, exactly the way a hash table maps a key to a value. In the POC every limit the shell needs — setpoints, valve positions, the Shield's cap, the uncertainty threshold — lives here. The AI never produces any of them.



Figure 10.1 — From a question to a fact: normalise, hash, find the version valid at time t , return the vector.

Step 1 — normalise

Two questions that mean the same thing must produce the same key. The book is precise about this: normalisation must be fixed and documented, because a single differing character produces a completely unrelated hash. The POC's rule is deliberately minimal: **trim surrounding whitespace and convert to lower case** (invariant culture). “*High Pressure Relief Setpoint in bar*” and “*high pressure relief setpoint in bar*” are therefore the same question. Anything richer (collapsing inner spaces, synonyms, units) would be a design decision to document and version like any other fact.

Step 2 — hash

The normalised text is encoded as UTF-8 and hashed with **SHA-256**, giving a 64-character hexadecimal key. SHA-256 is deterministic: the same bytes always give the same 256 bits, on any machine, in any year. The test suite checks the book's own worked example:

```

"what is the maximum allowed discharge pressure for pump p-101?"
-> f20ab319ace115290fd196a1c1d5b574e60ed0b39692add217c4ff0bcf03e574

"high pressure relief setpoint in bar"
-> 6294ef1683122a9127e1ebbe303962e952ec6ebbc9b605602b40115eb11c2a70
  
```

How safe is it to treat different hashes as different questions? The book's §24.2.1 answers with the birthday bound: for a 256-bit hash about 4×10^{38} distinct inputs are needed before a collision becomes as likely as not. At a physically absurd 10^{18} hashes

per second that is roughly 10^{13} years — some 920 times the age of the universe. The vault's correctness rests on that number, not on a hope that “hashes are usually fine”.

Step 3 — look up the version valid now

A key does not point to one value but to a **history**: v1, v2, ... Each version carries a Determinism block with ValidFrom (inclusive) and ValidTo (exclusive, empty while current). The lookup takes a moment in time and returns the version valid at that moment. The next chapter is about why this matters.

What a fact looks like

```
ILMVector
VectorId      9f72e7a1-...      unique, never reused
Embedding     [ 8.0 ]          the value (one number in this POC)
Determinism
  IsDeterministic true
  Version        "v1"
  ValidFrom      2026-01-01T00:00:00Z
  ValidTo        null           (open: this is the current truth)
  ApprovedBy     "Shift Safety Engineer"
  PointerTo      null           (Chapter 12)
  DerivedFrom    [ ... ]        (Chapter 13)
  DerivedBy      "minimum"
```

The shape is the book's ILMVector from Chapter 10, kept on purpose. In an Industry Language Model the embedding is a long vector; here it is one number. Everything else — identity, determinism, validity, approval — is identical, which is why the mechanisms in this POC transfer directly to the language-model setting the book describes.

The eleven facts of the demo

Question (normalised)	Value	Used by
maximum safe valve opening for ai-proposed commands	0.75	Shield
maximum ai prediction uncertainty as 95% half-width	0.20	Uncertainty escalation
maximum safe reading of pressure transmitter pt-1 in bar	8.4	Derived setpoint
maximum safe reading of pressure transmitter pt-2 in bar	8.0 (7.6 from v2)	Derived setpoint
maximum safe reading of pressure transmitter pt-3 in bar	8.2	Derived setpoint
high pressure relief setpoint in bar	min of the three	Pressure island
valve opening for high pressure relief	1.0	Pressure island
maximum coolant temperature in celsius	90	Structural island
valve opening for structural cooling	0.6	Structural island
valve opening for radiation containment	0.0	Radiation island
high pressure relief setpoint for backup line b in bar	→ pointer	Scenario 7

KEY IDEA · Why a hash and not just the text?

A hash gives a fixed-size, collision-resistant identifier that is identical everywhere. It can be logged, compared and signed without carrying the text, and it makes the formal guarantee of §24.2.3 trivial: the vault is the composition $A \circ H$ of a hash function H and a finite lookup table A , and equal hashes give equal answers because A is a function. The engineering effort goes into making the hypotheses true — H pure, A unambiguous at query time — not into the proof.

11 Time: versions, validity and point-in-time answers

A safety limit is never edited. It is superseded. (Book, §12.4.4 and §24.4.1.)

What happens when an engineer changes a setpoint? The tempting answer — overwrite the value — destroys the one thing an incident investigation needs: what the system believed *at the time*. The book therefore mandates an **append-only** discipline. To update a fact:

- 1 set the current version's ValidTo to the effective date of the change;
- 2 register a new vector with a new VectorId, an incremented Version and ValidFrom equal to that same date;
- 3 record who approved it. In the POC an update without an approver is rejected outright.

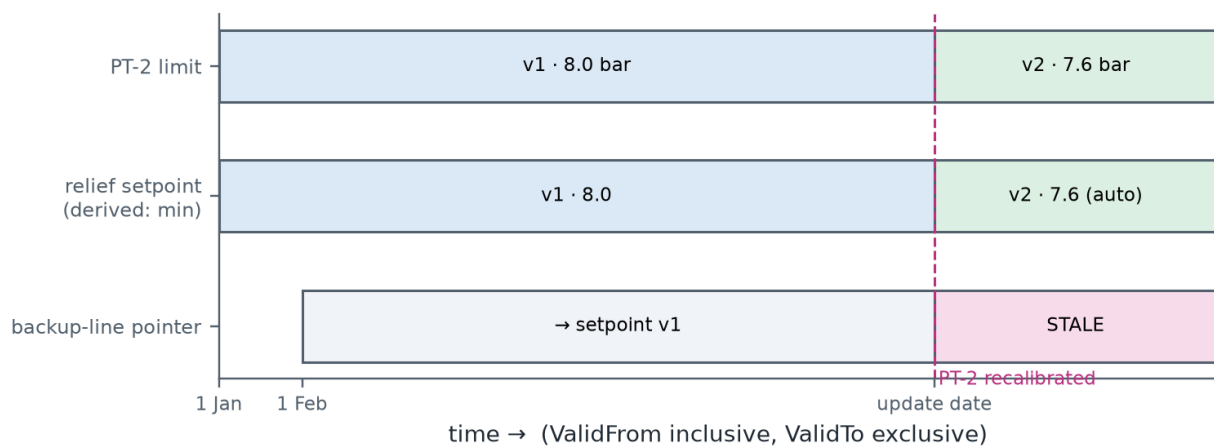


Figure 11.1 — Validity intervals in the demo. Recalibrating PT-2 closes its v1 and opens v2; the derived relief setpoint follows automatically (Chapter 13); the backup-line pointer, which pointed at the old setpoint version, becomes stale (Chapter 12).

Formally (§24.4.1) the validity intervals of all versions of one question must **partition time** without gaps or overlaps, so that “what was the answer on this date?” always has exactly one answer. The POC enforces the necessary ordering: a new version must start strictly after the current one. Lookups take an explicit instant, so the Arbiter asks for the facts valid at the moment of the control cycle, and a test can ask what the answer was last February.

OUR APPROACH · Rejecting retroactive updates

We added one rule the book implies but does not spell out: an update is rejected if it would start before a derived island that depends on it (Chapter 13). Such an update would make that island stale for its whole validity period. It is cheaper to refuse the update and ask for a correct effective date than to discover the inconsistency during an incident.

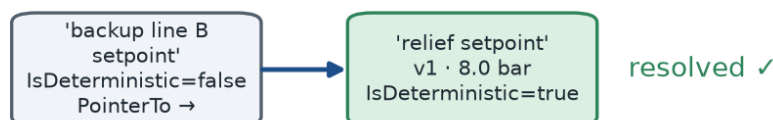
TRY IT · Ask the past

In the tests, `Register_UpdateIsAppendOnlyAndKeepsHistoryQueryable` registers a SCRAM setpoint of 320 on 1 January and 315 on 1 June, then asks for the value one day before 1 June (320) and on 1 June (315). Change the dates and watch which assertions fail.

12 Intra-Vector Routing: facts that point to other facts

The third deterministic mechanism of §12.3.3: a vector may defer to another vector instead of holding a value.

Real plants are full of facts that are, by specification, the same as some other fact: a backup line designed to the primary line's rating, a spare pump with the duty pump's limit. Copying the value is dangerous — the copies drift apart. The book's answer is a **pointer**: a vector whose Determinism block says `IsDeterministic = false` and `PointerTo = <another VectorId>`. Resolution follows the pointer and defers to the target's determinism.



Failure modes: none of them may fall back to a stochastic answer



Figure 12.1 — Scenario 7's pointer and the four ways a chain can fail. Every failure is loud; none falls back to a stochastic answer.

The resolver

```

resolve(start):
    current = start; visited = { start.id }
    while current is not deterministic and current points to next:
        if next already visited      -> fail "cycle"
        if visited count > 8         -> fail "too many hops"
        current = store.get(next)    or fail "dangling pointer"
    return current                  # deterministic, or an ordinary vector

resolveUnderLock(start, lock, t):
    r = resolve(start)
    if lock engaged and r is not deterministic -> DeterminismViolation (Human-in-the-Loop)
    if r is deterministic and not valid at t    -> fail "stale pointer"
    if r is derived and any input not valid at t -> fail "stale derived island"
    return r

```

The first function is the book's `VectorResolver`; the second its `GuardedResolver`, extended by us with the two time checks. The book is explicit that the cycle and dangling-pointer guards are “not optional hardening”: a cycle would loop forever, and a dangling pointer must fail loudly rather than silently fall back to an ungrounded answer. Every safety limit the POC reads goes through `resolveUnderLock` with the lock engaged, so a pointer chain that does not end in a valid deterministic fact can never reach an island.

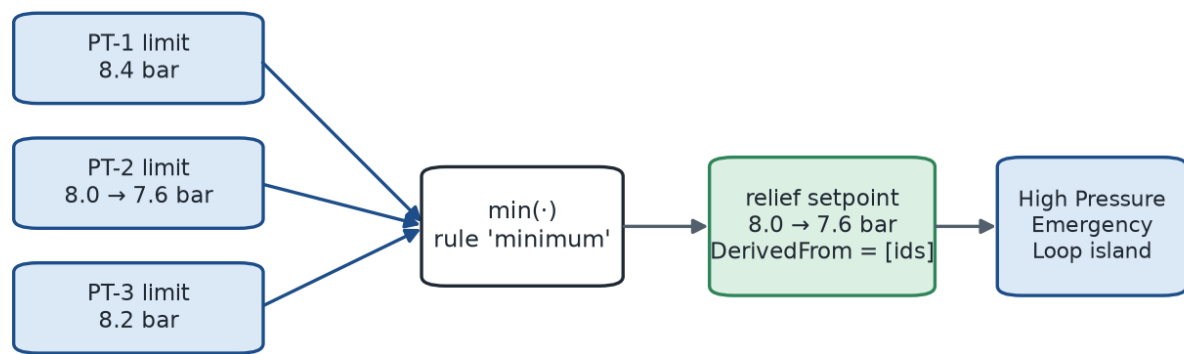
OUR APPROACH · Stale pointers

A pointer names one specific *version* of its target, by `VectorId`. When the target gets a new version, should the pointer silently follow? We decided no. The engineer who approved “backup line B uses the primary setpoint” approved it against a particular value; if that value changes, the pointer is marked stale and resolution fails until a human re-registers it. This is conservative, and it is exactly the §12.4.4 principle — no safety fact changes without human sign-off — applied to routing. Scenario 7 followed by scenario 8 shows it happening.

13 Derived islands: facts computed from facts

Three redundant transmitters, three certified limits, one safe operating limit: the minimum. (Book, §24.4.1 and §24.5.)

The book's deep-dive chapter introduces a new kind of island: one whose answer is a **pure function** of other islands' answers. Its example is three redundant pressure sensors with certified maxima of 45.0, 42.5 and 44.0 bar; the system's safe limit is their minimum, 42.5 bar. Because min is an ordinary mathematical function, a function of deterministic inputs is itself deterministic — the one-line argument of §24.2.3 again.



a new PT version re-registers the derived island automatically (cascading)

Figure 13.1 — The POC's relief setpoint is a derived island over three transmitter limits, and the pressure-relief island reads it like any other fact.

In the POC the relief setpoint is exactly such an island: $\min(8.4, 8.0, 8.2) = 8.0$ bar. The derived vector records `DerivedBy = "minimum"` and `DerivedFrom =` the VectorIds of the three inputs it was computed from. That second field is what makes the value auditable: you can always tell which input versions produced it.

Staying current: automatic recomputation

The book names the failure mode plainly in its Honest Boundary: a derived island is only as current as the last time it was recomputed, and nothing in its sample code detects that an input has changed (§24.6). It also says what should happen: the §12.4.4 update discipline, *applied automatically rather than manually*. The POC implements both halves:

- **Automatic recomputation.** When any input question receives a new version, every derived island that lists it is re-registered from the same effective date, and so on down the chain (a derived island of a derived island is recomputed in cascade). The new version's approver reads, for example, *"Instrumentation Engineer (auto-recomputed after '...pt-2...' changed)"*, so the audit trail shows both who changed the input and that the derivation followed automatically.
- **Stale detection.** If an input changes by a route that does not trigger recomputation — typically because the derived island depends on a *pointer* whose target changed — the resolver checks every `DerivedFrom` id against the query time and fails loudly with "stale derived island".

Scenario 8 shows the happy path: PT-2 is recalibrated from 8.0 to 7.6 bar, the setpoint becomes $\min(8.4, 7.6, 8.2) = 7.6$ bar as version v2, and a reading of 7.8 bar — harmless a moment earlier — now triggers the pressure-relief island.

OUR APPROACH · Rules by name, not by code

A derived island stores the *name* of its rule ("minimum", "maximum", or one registered by the application), not a piece of code. That keeps the rule persistable and reviewable: the vault file (Chapter 19) says *what* function produced a value, and reloading a vault with an unknown rule fails rather than guessing.

HONEST BOUNDARY · Island sprawl

The book's second warning in §24.6 applies fully: as the number of facts and derivations grows, the human review burden grows with it, and a registry too large for any one reviewer to understand is a governance failure however sound each entry is. Automatic recomputation reduces clerical work; it does not reduce the need for someone to understand the graph.

14 Protection islands and the Shield

Facts become behaviour: interlocks that take control of the valve, and a cap on everything the AI proposes.

So far the vault holds numbers. A **protection island** turns numbers into a rule: a condition on the sensor reading, an enforced valve position, a priority ring, and the list of vault facts it was built from. The island catalogue is rebuilt on every cycle from the facts valid at that instant, so an approved change to a setpoint changes the island without touching any code.

Island	Ring	Condition	Enforced opening	Facts it reads
High Pressure Emergency Loop	CriticalSafety	$P \geq \text{relief setpoint}$	relief opening (100%)	relief setpoint (derived), relief opening
Radiation Leak Containment Boundary	CriticalSafety	leak detected	containment (0%)	containment opening
Structural Thermal Protection	Structural	$T \geq \text{coolant limit}$	cooling (60%)	coolant limit, cooling opening

OUR APPROACH · Islands are always on

In the POC's first version, islands were created only when the operator notes contained certain keywords; at 8.5 bar without a keyword the AI ran unchecked. The book is clear that hard temperature and pressure limits are non-negotiable and enforced regardless of what any model recommends (§17.6), so the islands are now evaluated on every single cycle. Keywords now only control the Causality Lock (Chapter 16), which is a request for a *stronger* guarantee, never a precondition for safety.

The Shield

When no island is triggered the AI's proposal may be used, but never as is. The **Shield** of §17.4 wraps it in a non-learned layer: $\text{command} = \min(\text{proposal}, \text{maxSafe})$, where maxSafe is the vault fact “maximum safe valve opening for ai-proposed commands” (0.75). The book describes the effect precisely: the model is free to propose 85% again on the next step, and even to learn over time that such proposals are never applied, but the plant is never exposed to 85% while it learns. The audit record distinguishes **AiApproved** (the proposal was within bounds) from **AiShielded** (it was capped), and

keeps the original proposal.

KEY IDEA · Shielding is Deterministic Islands applied to actions

The book's own observation (§17.4): the shield's safe range is exactly the kind of static, auditable, non-stochastic constraint the Arbiter enforces for facts. The same vault, the same versions, the same approvals — applied to a command instead of an answer.

15 The Arbiter: one control cycle, step by step

Every guarantee in this guide meets in one place. This chapter walks through it in the order the code executes it.

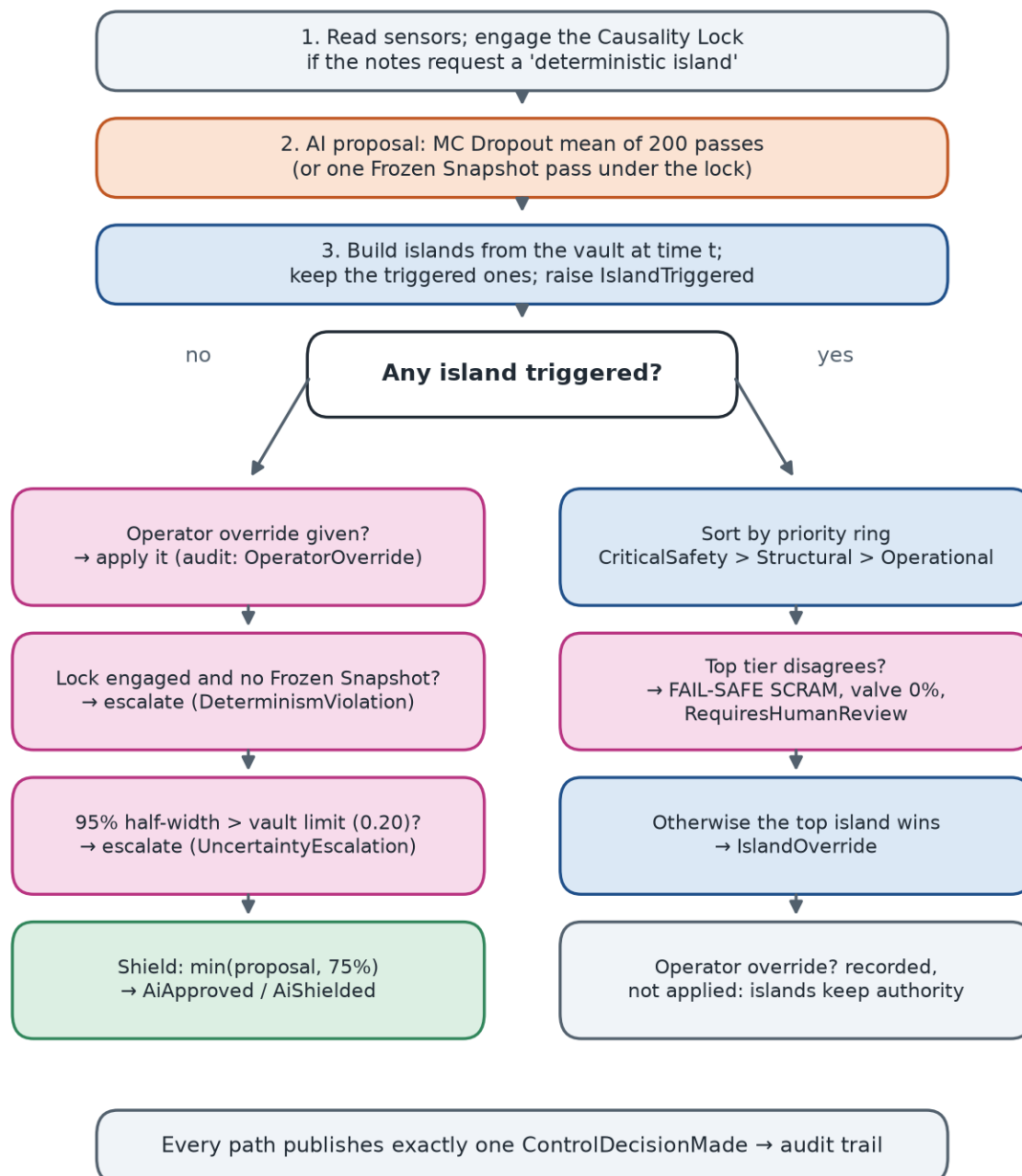


Figure 15.1 — One call to `Execute(reading, override)`. Exactly one `ControlDecisionMade` event is published on every path.

Before the branch

- 1 Validate the override**, if any: a named operator, a reason, an opening between 0 and 1. An invalid override is rejected before anything else happens, and no decision is recorded.
- 2 Engage the Causality Lock** if the operator notes contain “deterministic island”.
- 3 Compute the AI proposal**. Normally the MC Dropout mean of 200 passes with its 95% band; under the lock, if a Frozen Snapshot is configured, one reproducible pass instead.

- 4 **Build and evaluate the islands** from the vault at the cycle's timestamp, and publish an IslandTriggered event for every fact behind every triggered island.

Branch A — no island triggered

The AI is a candidate, and four questions are asked in order. Is there an operator override? Then it is applied (Chapter 17). Is the lock engaged without a Frozen Snapshot? Then the system refuses to answer stochastically and escalates. Is the 95% band wider than the vault's limit? Then it escalates for uncertainty. Otherwise the Shield caps the proposal and it is applied.

Branch B — at least one island triggered

The AI's proposal is recorded for the audit trail but plays no further role. Triggered islands are sorted by priority ring. If the top ring contains islands with *different* demands — scenario 4, where high pressure wants the valve fully open and a radiation leak wants it fully closed — the Arbiter does not pick one. It commands a **fail-safe SCRAM** (valve to 0%, marked as scrambled) and flags the record for human review. Otherwise the top island wins.

Decision	When	Valve	Human review flag
AiApproved	No island; AI confident and within the Shield	AI mean	no
AiShielded	No island; AI confident but above 75%	75%	no
IslandOverride	An island (or several agreeing ones) triggered	island's value	no
Scram	Top-ring islands disagree	0% (scrammed)	yes
HumanEscalation	Lock without deterministic answer, or AI too uncertain	not changed	yes
OperatorOverride	Named operator command and no island triggered	operator's value	no

OUR APPROACH · Priority rings and SCRAM are an extension

The book's precedence rule (§12.4.3) is between *mechanisms* — Static Vault, then routed vector, then Frozen Snapshot — and it treats any disagreement between two deterministic sources as a data maintenance error for human review, never something the Arbiter quietly adjudicates. The POC keeps that spirit and adds a control-specific layer: when two safety interlocks of equal rank demand opposite actions, the only defensible automatic action is the fail-safe one, and the record is still flagged for a human. Both the README and the code comments say that this layer is ours.

16 The Causality Lock and Human-in-the-Loop

When someone asks for certainty, the system either provides it or says it cannot. It never pretends. (Book, §12.3.3.)

The **Causality Lock** is the book's name for an explicit request for determinism. Once engaged, every part of the answer must resolve to a deterministic source; there is no silent fallback to the stochastic generator anywhere in the path. When that is impossible, the system escalates to **Human-in-the-Loop**: it reports that it cannot answer deterministically, rather than quietly relaxing the guarantee it was asked for.

Situation (lock engaged)	Result
An island decides the cycle	The island's command — islands are deterministic.
No island, Frozen Snapshot configured	One reproducible pass of the frozen core, then the Shield (scenario 6).
No island, no Frozen Snapshot	DeterminismViolationException; decision HumanEscalation; AI not applied (scenario 5).
Query path, vault hit	The resolved fact (following pointers).
Query path, only a non-deterministic candidate	DeterminismViolationException (scenario 7, second half).

In the POC the lock is engaged when the operator notes contain the phrase “deterministic island”. Our first version also reacted to “retrieve”, which the book never uses for this purpose; it was removed when the POC was aligned with the text. The exact trigger is less important than the principle: engaging the lock can only make the system *more* conservative.

Two kinds of escalation

Escalation is represented as an exception with a common base type, `HumanEscalationException`, and two concrete kinds: `DeterminismViolationException` (the lock could not be honoured) and `UncertaintyEscalationException` (the AI's 95% band is too wide, Chapter 8). Both are published as a `HumanEscalation` decision with no `valve` value and the `human-review` flag set *before* the exception is thrown, so an escalation is never lost even if the caller ignores it.

KEY IDEA · An escalation is a decision, not an error

It is tempting to treat “I cannot answer” as a failure. In a safety architecture it is the correct output for a well-defined class of inputs, and it is tested, audited and counted like any other outcome. The Web API reflects this: an escalation returns HTTP 200 with *applied: false*, not an error code.

17 Operator override and the limits of human authority

Rule 5 of the Neural Constitution: no fully autonomous action without a human retaining override authority. But authority over what?

The book's Neural Constitution (§26.4) includes the rule that a human always retains override authority. The POC implements this as an **operator override**: a command given by a named operator, with a stated reason, carried into the control cycle. It replaces the AI's proposal, and it is the natural answer to an escalation: the system says “I cannot decide this safely”, an operator checks the local gauges and decides.

The harder design question is what the override may *not* do. The book is equally clear that in reactor control the hard limits are enforced by certified safety systems that keep final physical authority regardless of what any model recommends (§17.6). We read that as applying to people too, in this narrow sense: a triggered safety island or a SCRAM is itself the certified safety function, and an operator command does not bypass it in software.

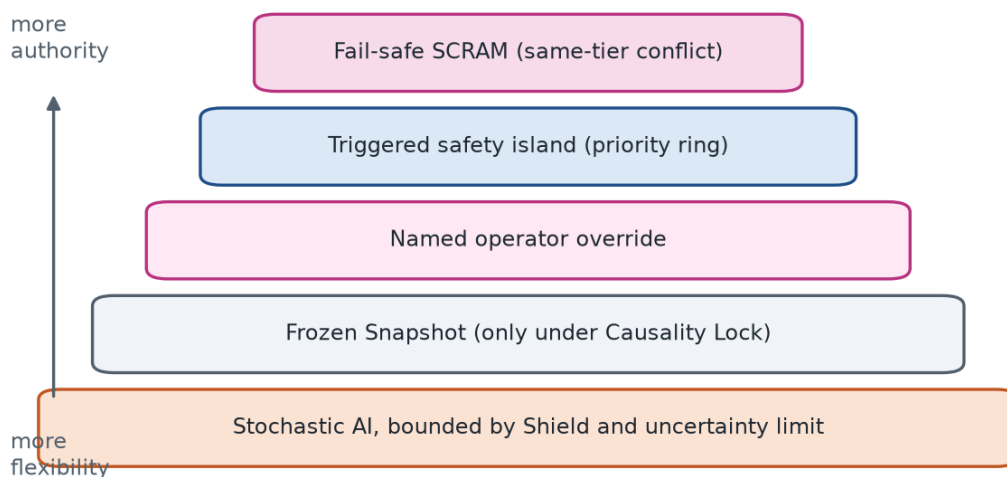


Figure 17.1 — The authority ladder used by the Arbiter. Higher rows override lower rows; lower rows are more flexible.

Situation	Override applied?	What is recorded
No island; AI would be approved	Yes	OperatorOverride with operator, reason and the AI's proposal
No island; lock or uncertainty escalation	Yes — this is how an escalation is answered	OperatorOverride
Island triggered	No	IslandOverride + “Operator override (...) not applied: safety island ... retains authority”
SCRAM	No	Scram + “... not applied: the fail-safe SCRAM retains authority”

Situation	Override applied?	What is recorded
Invalid (no name, no reason, outside 0-1)	Rejected	Nothing — the request is refused before a decision

OUR APPROACH · Why the operator is not bound by the Shield

The Shield's limit is, by its own name in the vault, the maximum safe opening for *AI-proposed* commands. It exists because the AI can be wrong in ways nobody reviewed. A named operator acting on local information is a different kind of authority, so the POC lets the override use the full 0-100% range — while triggered islands still take precedence. A real plant would add procedures, permissions and perhaps a second pair of eyes; the audit trail records who did what and why.

PART IV

Trust infrastructure

A decision nobody can reconstruct is a decision nobody can defend. Events, audit records, tamper-evident storage, a written constitution and a thin API.

18 Domain events and the audit trail

The Arbiter announces what happened; it does not know who is listening.
(Book, §22.7, §23.4, §19.5.)

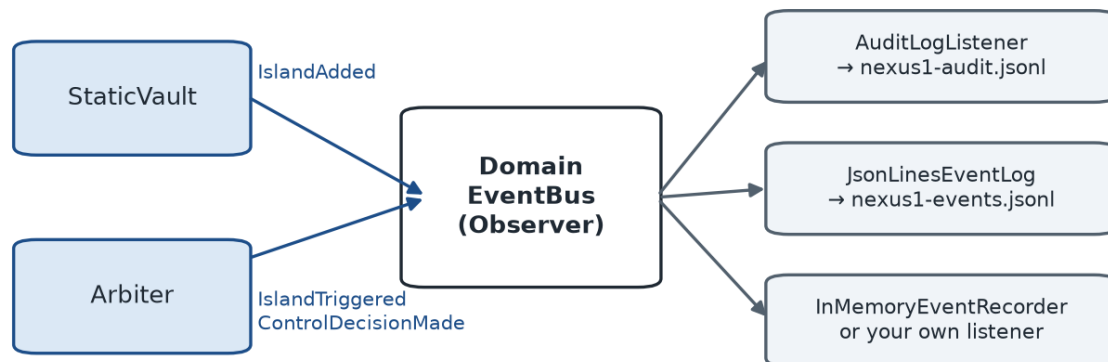


Figure 18.1 — Publishers, the event bus and listeners. Adding a new consumer — a dashboard, an alarm, a compliance export — does not touch the Arbiter or the vault.

The book's DDD chapters give auditing a precise place. The domain raises **events**; infrastructure **listens**. In its words, a subscriber can listen for one event type and build a complete, append-only audit trail of every deterministic answer, “without the Arbiter itself needing to know that auditing exists” (§22.7). The POC has three events:

Event	Published by	When	Carries
IslandAdded	Static Vault	Every new version: value, pointer, derived island, automatic recomputation	hash, question, the full vector, derivation rule and inputs
IslandTriggered	Arbiter	A vault fact governed a decision (behind a triggered island, or a vault hit in Answer)	hash, question, VectorId, version, time
ControlDecisionMade	Arbiter	Exactly once per control cycle, on every path including escalations	the complete audit record

Anatomy of an audit record

```
{ "Timestamp": "2026-09-26T11:23:39Z",
  "Reading": { "TemperatureCelsius": 40, "PressureBar": 8.5,
    "OperatorNotes": "Critical failure. ...", "RadiationLeakDetected": true },
  "CausalityLockEngaged": false,
  "AiProposal": 0.649, "AiOrigin": "AI Stochastic Core (MC Dropout mean of 200)",
  "AiUncertainty95": 0.29,
  "TriggeredIslands": [
    { "Name": "High Pressure Emergency Loop", "Priority": "CriticalSafety", "EnforcedOutput": 1,
      "Sources": [ "high pressure relief setpoint in bar = 8 (v1, derived: minimum of 3 in puts, ...)",
        "valve opening for high pressure relief = 1 (v1, approved by ...)" ] },
    { "Name": "Radiation Leak Containment Boundary", "Priority": "CriticalSafety", "EnforcedOutput": 0, ... } ],
  "Decision": "Scram", "FinalValveOpening": 0, "Origin": "FAIL-SAFE EMERGENCY SCRAM",
  "RequiresHumanReview": true,
  "Reason": "Conflicting demands at the same priority level (CriticalSafety): ...",
  "Operator": null }
```

Read the record from top to bottom and you have the whole story: what the plant looked like, whether determinism was requested, what the AI wanted and how sure it was, which safety rules fired and on the basis of which fact versions and approvals, what was decided, and whether a human must look at it. This is §19.5's point made concrete: a complete audit trail is simply the discipline of logging the decisions the architecture is already making, alongside the version metadata of the facts it used.

KEY IDEA · Log inputs, not just outputs

An audit trail that records only “valve set to 0%” cannot answer why. The expensive part — and the valuable part — is recording the inputs to the decision with their provenance: the AI's proposal even when it was overridden, and the exact versions of the facts that overrode it.

19 Persistence and the hash chain

A vault that forgets on restart is not a vault. A vault file anyone can edit silently is not evidence. (Book, §23.2, §19.5.)

The Static Vault sits behind a narrow repository interface with two operations: *save one record* and *return every record in order*. That is the whole contract, and it matches the vault's own append-only discipline. The POC ships an in-memory implementation (tests) and a JSON Lines file (one JSON object per line, never rewritten).

Save first, then apply

Every new version is written to the repository **before** it takes effect in memory. If the write fails — a full disk, a permissions error — the in-memory vault is unchanged and the caller sees the error. The alternative order would allow a limit to be in force that exists nowhere on disk, which is precisely the kind of undocumented state the architecture exists to prevent.

Replay on start-up

When a vault is opened on an existing file it replays every record exactly: same VectorIds, same versions, validity intervals closed by their successors. Pointers and derived islands therefore resolve after a restart exactly as before, and automatic recomputation keeps working. Replay raises no events and recomputes nothing — the file already contains every version, including the automatic ones. Each record is checked against what has been replayed so far, and an inconsistent file is refused with the record number: missing approver, skipped version, dates out of order, a pointer or derivation input that was not saved earlier, or an unknown derivation rule.

The hash chain

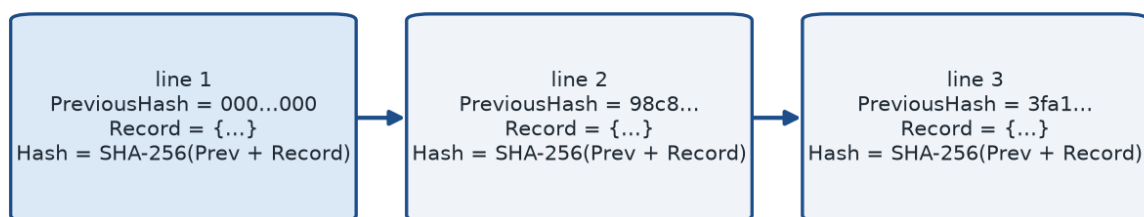


Figure 19.1 — Each line stores the hash of the previous line. Changing any record changes its hash and breaks every link after it.

Each line of `nexus1-vault.jsonl` has three fields: `PreviousHash`, `Hash` and `Record`, with $Hash = SHA-256(PreviousHash + Record)$ and the first `PreviousHash` equal to 64 zeros. The record is embedded verbatim, so the bytes that were hashed are the bytes on disk. On load, the chain is verified line by line:

Tampering	Detected as
A value edited in place (e.g. 8 → 9)	line N: hash mismatch; this record was altered after it was saved
A line removed from the middle	line N: hash chain broken; a record before this line was removed, added or reordered
Two lines swapped	line 1: hash chain broken
Malformed JSON	line N: not a valid island record

HONEST BOUNDARY · What a chain cannot see

Lines cut off the *end* of the file leave a shorter but valid chain, and someone with write access can rewrite the whole file with a freshly computed chain. The POC exposes the last hash (`HeadHash`, printed by scenario 10) so it can be recorded somewhere else — a shift log, a separate system, a signed receipt — and compared on the next start. Real systems anchor the head hash externally or sign records with a key; the POC stops at making the need visible.

20 The Neural Constitution

Not new enforcement, but legibility: one list of everything the system promises, each item traceable to the code that enforces it and the test that proves it.

(Book, §26.4.)

Every island, every shield and every precedence rule is, on its own, a technical mechanism. The book proposes naming them together as a **Neural Constitution**: the complete, explicit set of rules the system must never violate, whatever its stochastic parts learn. The book's version has five rules, each with an “EnforcedBy” reference. The POC keeps those five, adds three of its own, and adds one thing the book's sketch does not have: a **VerifiedBy** list of test names for each rule.

#	Rule	Enforced by	Source
1	A resolved deterministic fact is never overridden by a generated one.	Arbiter precedence; islands before AI; vault before candidates	Book
2	No control action reaches an actuator outside its certified safe range.	Shield (vault limit)	Book
3	A Causality Lock, once engaged, never silently falls back to a stochastic answer.	GuardedResolver, lock escalation	Book
4	Every registered fact's provenance and approval are permanently recorded.	Append-only versions; save-before-apply; hash chain	Book
5	No fully autonomous action is taken without a human retaining override authority.	Operator override; escalation; sign-off	Book
6	An AI command is never applied when its uncertainty exceeds the certified bound.	MC Dropout vs vault threshold	POC
7	A routed or derived fact is never used once the fact it rests on has changed.	Stale pointer / stale derived checks	POC
8	Conflicting safety demands at the same priority end in a fail-safe SCRAM, never a guess.	Priority rings, SCRAM	POC

A constitution that cannot drift

A list of promises in a document goes out of date the day the code changes. The POC's constitution is code, and a test reads it: for every rule, every cited test must exist in the test assembly as a real test method. Rename or delete a test that a rule relies on and the build goes red. When we tried citing a non-existent test on purpose, the failure message named the rule and the missing test. The program prints the constitution at start-up, and the Web API serves it at GET /constitution.

OUR APPROACH · From “EnforcedBy” to “VerifiedBy”

The book notes that the constitution itself should be governed like any other fact — reviewed on a cadence and changed only through an approved process (§26.5). Linking each rule to executable evidence is our contribution to that governance: a reviewer can go from a promise, to the mechanism, to a test that demonstrates it, to the green build that ran it.

21 The Web API

The thinnest layer of all: translate an HTTP request into a call on the application layer and translate the answer back. (Book, §23.3.3, §23.5.)

Endpoint	What it does	Notable responses
POST /control	One control cycle; optional operator override	200 with applied=false on escalation; 400 for an invalid override
POST /answer	A factual query with the §12.4.2 precedence	409 when determinism is required but impossible; isDeterministic=false on fall-through
GET /vault/history?query=...	Every version of one fact with validity, approver, pointer, derivation	404 if the fact is unknown
GET /vault/queries	The facts currently registered	
GET /constitution	The Neural Constitution with mechanisms and tests	

```
POST /control
{ "temperatureCelsius": 95, "pressureBar": 4, "radiationLeakDetected": false }

200 OK
{ "applied": true, "valveOpening": 0.6, "decision": "IslandOverride",
  "origin": "Deterministic Island: Structural Thermal Protection",
  "requiresHumanReview": false, "aiProposal": 0.32, "aiUncertainty95": 0.30,
  "triggeredIslands": [ "Structural Thermal Protection" ] }
```

Behind the endpoints sits one application-layer object that wires the event bus, reloads the hash-chained vault, commissions the default limits only if the file is empty, and processes requests one at a time — the domain model is not thread-safe, and the book's Honest Boundary in §22.8 names concurrency as one of the things a teaching model leaves out.

HONEST BOUNDARY · No endpoint writes a safety limit

It would take ten lines to add “POST /vault”. It is deliberately absent. The book lists authentication and authorisation on who may register an island among the prerequisites for any production use (§22.8), and an unauthenticated write endpoint for safety limits would undo every other guarantee in this guide. Limits change through code-reviewed commissioning, with named approvers, until proper identity and permissions exist.

PART V

Our approach

What we took from the book as written, what we deliberately changed, what we added — and why. This is the part to read if you need to judge the POC rather than learn from it.

22 From book to code: a traceability map

Book	Concept	In the POC	Status
§12.3.1	Static Vault, SHA-256 routing	StaticVault, normalisation, point-in-time lookup	As written
§12.3.2	Frozen Snapshot	FrozenSnapshotConfig, per-call seeding, weights hash check	As written
§12.3.3	Intra-Vector Routing, Arbiter, Causality Lock, HITL	VectorResolver, GuardedResolver, CausalityLock	As written + stale checks
§12.4.2	Arbiter.Answer precedence	CompleteSystemArbiter.Answer	As written
§12.4.3	Conflict resolution	Precedence + priority rings + SCRAM + review flag	Extended
§12.4.4	Append-only updates, human sign-off	Versioning, ValidFrom/ValidTo, mandatory approver	As written + retroactive check
§13.6	MC Dropout	PredictWithUncertainty, 200 passes, 1.96·s	Applied in the loop (ours)
§16.9	Thresholds as islands	Uncertainty limit as a vault fact	Applied
§17.4	Shield	Shield: min(proposal, vault limit)	As written
§17.6	Certified safety keeps final authority	Islands/SCRAM beat operator override	Interpreted
§19.5	Audit trail from Arbiter decisions	AuditRecord per cycle, JSONL	As written
§22.7, §23.4	Domain events, Observer	IslandAdded, IslandTriggered, ControlDecisionMade	As written + ControlDecisionMade
§23.2	Repositories	IIslandRepository, JSON Lines, replay with checks	As written + hash chain
§23.5	Minimal Web API	DeterministicIsland.Api	As written, read-only vault
§24.2.2	Floating-point order	Single-threaded fixed-order loop	As written
§24.4.1, §24.5	Derived islands	RegisterDerived, DerivedIslandFactory, auto-recompute	Extended
§26.4	Neural Constitution	8 rules, VerifiedBy tests, checked by a test	Extended

23 Design decisions and their rationale

Every non-obvious choice, stated once, with the reason. If you disagree with one, this is the list to argue with.

Decision	Rationale
Safety islands are evaluated on every cycle.	A safety limit that applies only when someone types the right word is not a safety limit (§17.6).
“retrieve” no longer engages anything.	The book engages the Causality Lock only on an explicit request for a deterministic island.
The AI's proposal is the MC Dropout mean, not a single pass.	A single pass is one random draw; the mean is the model's best estimate and comes with a confidence measure.
Uncertainty above the vault limit escalates.	A proposal the model itself disagrees about by more than ± 20 points should not move a valve unattended.
The uncertainty threshold lives in the vault.	It is an engineering decision with an owner and a history, not a hyper-parameter (§16.9).
Pointers do not follow a superseded target.	The approval was given against a specific value; a change requires a new approval (§12.4.4).
Derived islands recompute automatically, but updates cannot be retroactive.	Automatic where the book asks for it; refused where automation would create a stale period.
Priority rings and SCRAM on same-ring conflict.	Two equally authoritative safety demands cannot be reconciled by software; fail safe and escalate.
Operator override beats the AI, never an island or SCRAM.	Human authority over the model; certified safety functions keep physical authority (§17.6).
The operator is not bound by the AI Shield.	The Shield limits AI-proposed commands; a named, accountable person is a different authority.
Save before apply; replay without events.	No limit may be in force that exists nowhere on disk; restarting must not rewrite history.
Hash-chain the vault file and expose the head hash.	Make tampering evident, and make the remaining gap (truncation, full rewrite) explicit.
Escalation is a decision, not an error.	It is recorded, counted and returned (HTTP 200) like any other outcome.
No write endpoint for limits in the API.	Without authentication and authorisation, writing safety limits over HTTP would undo every guarantee.
The constitution cites tests and a test checks the citations.	Promises that cannot drift silently away from the code.

24 Honest boundaries

The book's standing rule: nothing in this book claims to be exact that is not. The same applies to this POC.

- **Islands guarantee reproducibility, not correctness.** A wrong value in the vault is reproduced faithfully forever — until a human corrects it (§12.5). Verification of the stored value is a human process, not a software property.
- **The network is hand-weighted and tiny.** Its uncertainty map illustrates the method; it says nothing about any real process.
- **MC Dropout measures self-disagreement.** A model can be confidently wrong, especially outside its training distribution.
- **200 passes are a sample.** The estimated half-width varies by about ± 0.01 – 0.02 between cycles; readings near the ± 0.20 limit may be approved on one cycle and escalated on the next.
- **The hash chain cannot detect truncation or a full rewrite** without an externally recorded head hash.
- **No authentication or authorisation.** Approver names are free text. Anyone who can run the code can “approve” a change.
- **No concurrency in the domain model.** The API serialises requests; a real service needs a proper concurrency design.
- **Island sprawl and review burden** grow with the number of facts and derivations (§24.6).
- **Not certified, not certifiable as is.** The disclaimer in the README applies without exception.

PART VI

Hands-on

Run it, read what it says, then change it. The eleven scenarios, a graded set of exercises, and where to go after this guide.

25 The eleven scenarios, explained

Run the demo with “dotnet run” in the DeterministicIsland folder and read along. Each scenario isolates one mechanism.

```
git clone https://github.com/gregory82gr/DeterministicIsland
cd DeterministicIsland/DeterministicIsland
dotnet run          # .NET 8 SDK or later
```

#	Scenario	Input	What happens and why	Read
1	Normal operation	60 °C, 5 bar, routine notes	No island is triggered. The AI proposes the MC Dropout mean (about 35%) with a 95% band of about ± 17 points, inside the ± 20 limit, and below the 75% Shield cap: AiApproved.	Chapters 7, 8, 14
2	High temperature	95 °C, 4 bar	$T \geq 90$ °C triggers Structural Thermal Protection; the valve goes to 60%. The output lists the two vault facts, with versions and approver, that the island used.	Chapter 14
3	Priority resolution	95 °C, 8.5 bar	Both the Structural and the CriticalSafety pressure islands trigger. The higher ring wins; the valve opens to 100%. Note that the relief setpoint is shown as <i>derived: minimum of 3 inputs</i> .	Chapters 13, 15
4	Deadlock → SCRAM	40 °C, 8.5 bar, radiation leak	Two CriticalSafety islands demand 100% and 0%. The Arbiter cannot reconcile them, commands a fail-safe SCRAM and flags the record for human review.	Chapter 15
5	Causality Lock → Human-in-the-Loop	60 °C, 5 bar, “...deterministic island.”	Determinism is requested, no island applies and no Frozen Snapshot is configured. The system refuses to answer stochastically and escalates; the AI command is not applied.	Chapter 16
6	Causality Lock → Frozen Snapshot	same reading, frozen core (seed 42)	The same request now has a deterministic mechanism: the frozen network. Two calls give 41.7% and 41.7%, identical to the last bit.	Chapter 9
7	Intra-Vector Routing	query: backup line B setpoint	The backup line's setpoint is registered as a pointer to the primary setpoint and resolves to 8 bar. Then an ordinary vector from a pretend RAG lookup, without a Determinism block, is rejected under the lock.	Chapter 12
8	Derived island	PT-2 recalibrated to 7.6 bar; reading 60 °C, 7.8 bar	The relief setpoint is recomputed automatically from 8.0 to 7.6 bar (v2). A 7.8 bar reading, harmless before the change, now triggers the pressure-relief island.	Chapter 13
9	Uncertain AI	85 °C, 7 bar	No island applies, but the network disagrees with itself by about ± 29 points (95%). The command is not applied; the cycle escalates for uncertainty.	Chapter 8

#	Scenario	Input	What happens and why	Read
10	Persistence	reopen the vault from its file	A second vault is opened from nexus1-vault.jsonl. The hash chain verifies; it holds the same 13 versions of 11 facts with the same ids; the setpoint is still 7.6 bar (v2).	Chapter 19
11	Operator override	scenario 9's reading, then scenario 4's	The operator answers the uncertainty escalation with 55%, which is applied. The same operator then tries 100% during the SCRAM; the command is recorded as <i>not applied</i> .	Chapter 17

What the console actually prints

An excerpt of one real run (paths shortened). Numbers from the stochastic core vary slightly from run to run; everything deterministic is identical every time.

```

--- Scenario 1: Normal Operation (AI Decides, Bounded by the Shield) ---
[INFO] Safe Zone: No islands triggered. Executing: AI Stochastic Core (MC Dropout mean of...
--- Scenario 4: The Ultimate Deadlock (Same Priority Conflict -> SCRAM) ---
[FATAL DEADLOCK] Multiple islands triggered at the SAME priority level (CriticalSafety) w...
!! Island: 'High Pressure Emergency Loop' demands Valve Opening -> 100.0 %
!! Island: 'Radiation Leak Containment Boundary' demands Valve Opening -> 0.0 %
[FAIL-SAFE] Arbiter cannot resolve conflict logically. Triggering immediate SCRAM shutdown...
[INFO] Safe Zone: No islands triggered. Executing: AI Frozen Snapshot -> 41.7 %
[INFO] Safe Zone: No islands triggered. Executing: AI Frozen Snapshot -> 41.7 %
--- Scenario 8: Derived Island (Relief Setpoint = Minimum of Three Transmitters) ---
[DERIVED] high pressure relief setpoint in bar = 8 (v1, derived: minimum of 3 inputs, app...
[DERIVED] high pressure relief setpoint in bar = 7.6 (v2, derived: minimum of 3 inputs, a...
--- Scenario 9: Uncertain AI Prediction (MC Dropout Interval Too Wide -> Human-in-the-Loo...
[HUMAN-IN-THE-LOOP] AI prediction 64.2 % ± 29.6 % (95%) exceeds the permitted ± 20.0 %.
--- Scenario 11: Operator Override (Answers an Escalation, Never Overrides a Safety Islan...
[OPERATOR OVERRIDE] Shift Operator A set the valve to 55.0 % (AI proposed 61.7 %): Answer...
[FATAL DEADLOCK] Multiple islands triggered at the SAME priority level (CriticalSafety) w...
!! Island: 'High Pressure Emergency Loop' demands Valve Opening -> 100.0 %
!! Island: 'Radiation Leak Containment Boundary' demands Valve Opening -> 0.0 %
[FAIL-SAFE] Arbiter cannot resolve conflict logically. Triggering immediate SCRAM shutdown...
[OPERATOR] Operator override (100.0 % by Shift Operator A) not applied: the fail-safe SCR...

```

After the run, three files sit next to the program: nexus1-vault.jsonl (recreated on every run, hash-chained), nexus1-audit.jsonl (one line per control decision) and nexus1-events.jsonl (every IslandAdded and IslandTriggered). Open them; they are the ground truth of what the system did.

26 Exercises: a graded path into the field

Each exercise is small, has a clear success criterion, and teaches one idea you will meet again in real systems. Run “dotnet test” after each change.

Level 1 — Observe

- 1 Run the demo three times. Which numbers change and which never do? Explain each difference using Chapters 7–9.
- 2 Open `nexus1-audit.jsonl` and, for scenario 4, reconstruct the decision using only the file.
- 3 Compute by hand the network output for 30 °C, 5 bar with all neurons kept. Compare with Chapter 6's method and with the table in Chapter 8 (about 23%).
- 4 Hash a question of your own with any SHA-256 tool after normalising it. Check that capitalisation and surrounding spaces do not change the key, but an inner double space does.

Level 2 — Change a fact, not the code

- 1 Register a new version of the Shield limit (0.60) dated after 1 January. Which scenario's output changes? Is the old value still visible in the history?
- 2 Raise the uncertainty limit to 0.30. Which scenario stops escalating? Why is this a decision for an engineer and not for the model?
- 3 Recalibrate PT-3 to 7.0 bar. Predict the new relief setpoint before you run it, then check the `IslandAdded` events for the automatic recomputation.
- 4 Try to register an update dated before the derived setpoint's current version. Read the error and explain what problem it prevents.

Level 3 — Extend the architecture

- 1 Add an *Operational*-ring island, e.g. “close the valve to 20% below 2 bar”. Write a test where it competes with the Structural island and check the priority ring.
- 2 Register a derived island using “maximum”. Where would a maximum, rather than a minimum, be the safe choice? (Hint: think about minimum cooling flow.)
- 3 Add a fourth domain event, for example `ShieldApplied`, and a listener that counts how often the AI was capped. Do not change the Arbiter's decisions.
- 4 Add a rule to the Neural Constitution with a test that verifies it. Then rename the test and watch the constitution test fail.

Level 4 — Toward a real system

- 1 Replace the hand-set weights with weights trained on synthetic data (book, Chapter 7). What must be true of the new network before it may be used under a Frozen Snapshot?
- 2 Record the vault's head hash in a separate file on every start and refuse to start if the file was truncated. What new failure mode have you created, and how would operators recover from it?
- 3 Design (on paper) authentication for vault updates: who may approve which facts, and how would the approver's identity be bound to the record instead of free text?
- 4 Sketch how the query path (Answer) would sit in front of a real language model, following the book's §12.5: the model's open-ended text stays stochastic, every numeric limit it mentions is resolved through the vault.

TRY IT · A good habit

Before every change, write down what you expect the audit trail to show. After the change, compare. The gap between expectation and record is where understanding happens — and in a real plant, where incidents are prevented.

27 Where to go next

This guide covers the parts of the book that the POC implements. The book itself goes much further: information theory and the Shannon limit as the forerunner of learning (Part I), the full path from perceptron to Transformer (Part II), the Industry Language Model and its evaluation, ethics and governance (Part III), applications in predictive maintenance, anomaly detection, control and decision support (Part IV), and the formal treatment of Deterministic Islands (Part VI).

If you want to...	Read in the book
understand why neural networks are stochastic in the first place	Chapters 4–8
see the three deterministic mechanisms in full	Chapter 12
measure uncertainty and evaluate models honestly	Chapter 13
apply islands to anomaly thresholds and control loops	Chapters 16–17
build a safety case and an audit trail regulators accept	Chapter 19
structure the software with DDD	Chapters 20–23
see the mathematics behind the guarantees	Chapter 24
think about the future: Neural Constitution, quantum networks	Chapter 26

The NEXUS-1 series

From Stochastic Chaos to Deterministic Certainty is one of **22 books** in the NEXUS-1 series by Grigorios Agathangelidis. The series began with a single experiment, **NEXUS-1 Phase 0**, an interactive console for an educational digital twin of a nuclear plant that anyone can open at gregory82gr.github.io/Nexus-1-phase-0. From there it grew one open question at a time into physics, AI, data, architecture, formal methods, systems and frontend. It keeps one standing rule: nothing is stated as certain that has not been shown to hold in practice. The books are available at leanpub.com/u/grigorios-kyriakos-agathangelidis.

Theme	Book	In one line
Physics and engineering of the plant	<i>From Grid to Core</i>	The foundation of the series: from the 400 kV substation to the reactor core, with neutron kinetics and SCRAM.
	<i>From Queue to Core</i>	Stochastic queueing theory as a rigorous mathematical foundation for reactor kinetics, with C# implementations.
	<i>From Core to Quantum</i>	From the quantum crisis to the structure of the nucleus, and a complete quantum-circuit simulator in modelled C#.
Interpretable and controlled AI	<i>From Flood to Cause</i>	When 200 alarms fire at once: a deterministic causal graph finds the root cause; the LLM only explains it.
	<i>From Trial to Policy</i>	Reinforcement learning from scratch with a fully interpretable Q-learning agent: 175 numbers in an auditable table.
	<i>From Stochastic Chaos to Deterministic Certainty</i>	An industrial language model wrapped in proven deterministic boundaries, mapped to the EU AI Act. This POC's book.
Data architecture	<i>From Schema to System</i>	The complete schema atlas: 17 domains, 654 tables, with ER diagrams and verification queries behind the twin.
	<i>From Table to Twin</i>	The same schema built twice (Database First and Code First with EF Core) and a chapter reconciling them.
	<i>From Entity to Context</i>	Clean EF Core mapping of the schema: one configuration per entity instead of a giant OnModelCreating.
Domain-Driven Design	<i>From Domain to Twin</i>	DDD from scratch in plain language, applied to NEXUS-1: from entities and aggregates to the SQL schema.
	<i>From Context to Flow</i>	Advanced DDD patterns (context maps, sagas, outbox) through one flow that crosses nine bounded contexts.
Backend trilogy (.NET)	<i>From Blueprint to Core</i>	Domain and application layers without a database or web server: 17 contexts, 50 green tests in about 600 ms.

Theme	Book	In one line
	<i>From Core to Contract</i>	The core gets a database (EF Core, 654 tables) and a public API, with infrastructure strictly below the seam.
	<i>From Contract to Container</i>	Integration tests on a real SQL Server via Testcontainers, an outbox that survives a killed process, a CI gate.
Microservices	<i>From Flow to Services</i>	More than 70 chapters: microservices as a consequence of mature boundaries, not a starting point, and when distribution is not worth it.
	<i>From Services to Runtime</i>	Three owning services on a real .NET runtime blueprint: inbox/outbox, JWT, OpenTelemetry, Kubernetes.
Formal methods	<i>From Flow to Proof</i>	Architectural promises become proofs: state machines, TLA+, Petri nets, category theory; model versus implementation.
Systems trilogy (below .NET)	<i>From Runtime to Distribution — Volume I</i>	From C# to the hardware: CPU, kernel mode, threads, inside the CLR (JIT, Native AOT), memory and GC.
	<i>From Runtime to Distribution — Volume II</i>	The process boundary: concurrency, async/await as a state machine, IPC, TCP/TLS/gRPC and the “Boundary Ledger”.
	<i>From Runtime to Distribution — Volume III</i>	Production: containers, Kubernetes, SLIs/SLOs, canary deployments and practical observability.
Frontend (Angular)	<i>From File to Framework</i>	A 5,900-line single-file console becomes a real Angular application, and every screen is checked for “honesty”.
Retrospective	<i>From Certainty to Calibration</i>	The author revisits six of his own decisions: then → mechanism → lesson → now → what the correction risks.

Natural next reads after this guide: *From Flood to Cause* and *From Trial to Policy* apply the same principle as this POC (a deterministic core, with the learning or explaining component kept at its edge). *From Domain to Twin* and *From Blueprint to Core* go deeper into the DDD and layering used here. *From Flow to Proof* shows how promises like the Neural Constitution's become proofs. *From Certainty to Calibration* does for the author's own decisions what Part V of this guide does for the POC's.

Appendices

A map of the code, a map of the tests, and how to rebuild this guide.

A Code map

Where each concept of this guide lives. Paths are relative to the repository root.

Concept	Where
Sensor reading, command, islands, override	DeterministicIsland/domain/SensorReading.cs
ILMVector, Determinism block, VaultFact	DeterministicIsland/domain/ILMVector.cs
Neural network, MC Dropout, Frozen Snapshot check	DeterministicIsland/ProbabilisticCore/MiniNeuralNetwork.cs
Frozen Snapshot configuration	DeterministicIsland/Islands/FrozenSnapshotConfig.cs
Static Vault, versions, derived islands, replay	DeterministicIsland/Islands/StaticVault.cs
Repository, JSON Lines, hash chain	DeterministicIsland/Islands/IslandRepository.cs
Derivation rules, DerivedIslandFactory	DeterministicIsland/Islands/DerivationRules.cs, DerivedIslandFactory.cs
Pointer resolution, stale checks	DeterministicIsland/Islands/VectorResolver.cs
Causality Lock, escalation exceptions	DeterministicIsland/Islands/CausalityLock.cs
Safety facts and defaults	DeterministicIsland/Islands/SafetyLimits.cs
Protection islands	DeterministicIsland/Islands/IslandCatalog.cs
Shield	DeterministicIsland/Islands/Shield.cs
System Arbiter (control and query paths)	DeterministicIsland/CompleteSystemArbiter.cs
Audit record, audit logs	DeterministicIsland/Audit/AuditLog.cs
Domain events, bus, listeners	DeterministicIsland/Events/
Neural Constitution	DeterministicIsland/Governance/NeuralConstitution.cs
Demo scenarios	DeterministicIsland/Program.cs
Web API	DeterministicIsland.Api/
Tests	DeterministicIsland.Tests/
This guide's source	docs/source/

B Test map

100 automated tests (89 test methods; theories run once per case). Each file tests one concept.

Test class	Tests	What it demonstrates
StaticVaultTests	6	The book's SHA-256 example; normalisation; append-only versions; approver required
MiniNeuralNetworkTests	7	Stochastic vs frozen; bitwise reproducibility; MC Dropout interval
CompleteSystemArbiterTests	12	Shield, priority rings, SCRAM, Causality Lock, uncertainty escalation
IntraVectorRoutingTests	12	Pointer chains, cycles, dangling and stale pointers, Answer precedence
DerivedIslandTests	8	The book's min example; automatic and cascading recomputation; stale detection
DomainEventTests	8	Which events are raised, when, and to whom
IslandRepositoryTests	15	Replay, save-before-apply, consistency checks, hash chain tampering
OperatorOverrideTests	7 (11 cases)	Override applies, answers escalations, never beats an island or SCRAM
NeuralConstitutionTests	2 (9 cases)	Five book rules; every cited test exists
ApiTests	12	Endpoints, status codes, restart without re-commissioning, audit per cycle

C Rebuilding this guide

The guide is generated, not typed into a word processor, so that its numbers can never disagree with the code. The network in `docs/source/figures.py` mirrors `MiniNeuralNetwork.cs` exactly; the chapters are Python modules under `docs/source/`.

```

pip install reportlab matplotlib

# writes docs/NEXUS-1_Deterministic_Islands_Engineer_Guide.pdf
python docs/source/build_guide.py

# runs the demo first and embeds its fresh console output
python docs/source/build_guide.py --run

```

If you change the network's weights or the vault defaults, update the constants in `figures.py` and the fact table in Chapter 10, rebuild, and read the diff of the numbers. Treat the guide like the constitution: it is part of the evidence, and it should change in the same commit as the code it describes.

Licence

The code, the tests and this guide, including its source in `docs/source/`, are released under the MIT License (Copyright © 2026 Grigorios Agathangelidis; see the `LICENSE` file in the repository). The NEXUS-1 books quoted and cited here are separate works and are not covered by that licence. The software and this guide are provided “as is”, without warranty, and are not intended for any safety-related use.